MSE497

UGP presentation

Arvind Devkate & Anshul Suryawanshi

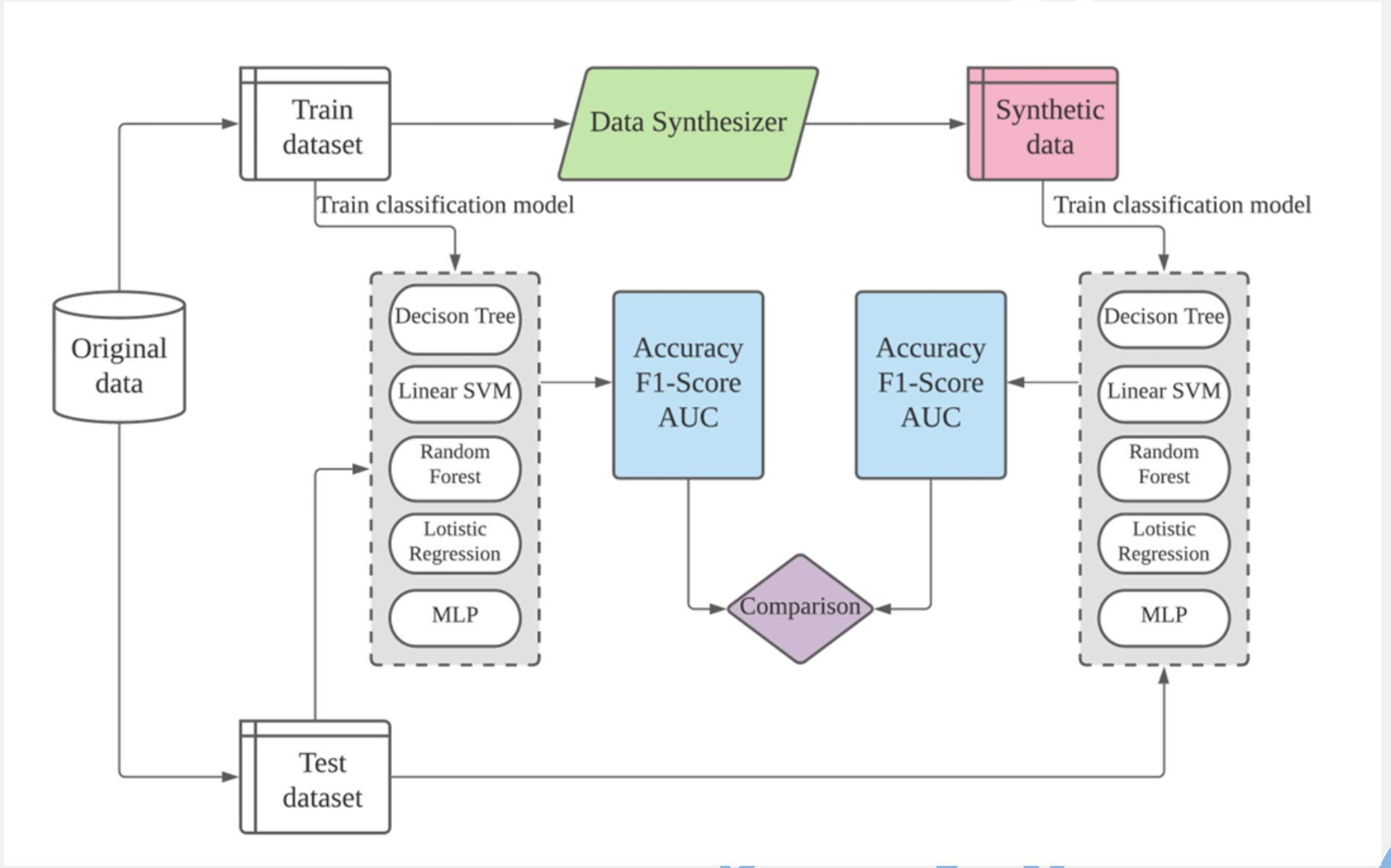
Prof Dr. Krishanu Biswas

High Entropy Borides Data

Parameters:

- rA/rC
- Simple diff-X
- Diff-Xmullican
- delta-Hf
- delta-Smix
- (Delta) misfit
- Possibility of formation (Single Phase - 1 Multiphase - 0)

Our methodology



Our methodology

01. Generating Synthetic Data for Borides using different methods
- Gaussian Mixture Models
- Condition Tabular GANs

02. Individual Feature analysis of the generated synthetic data and comparing it with the real data.

03. Training models (KNN, Random Forest) on the synthetic data and testing it on the previous test data sets

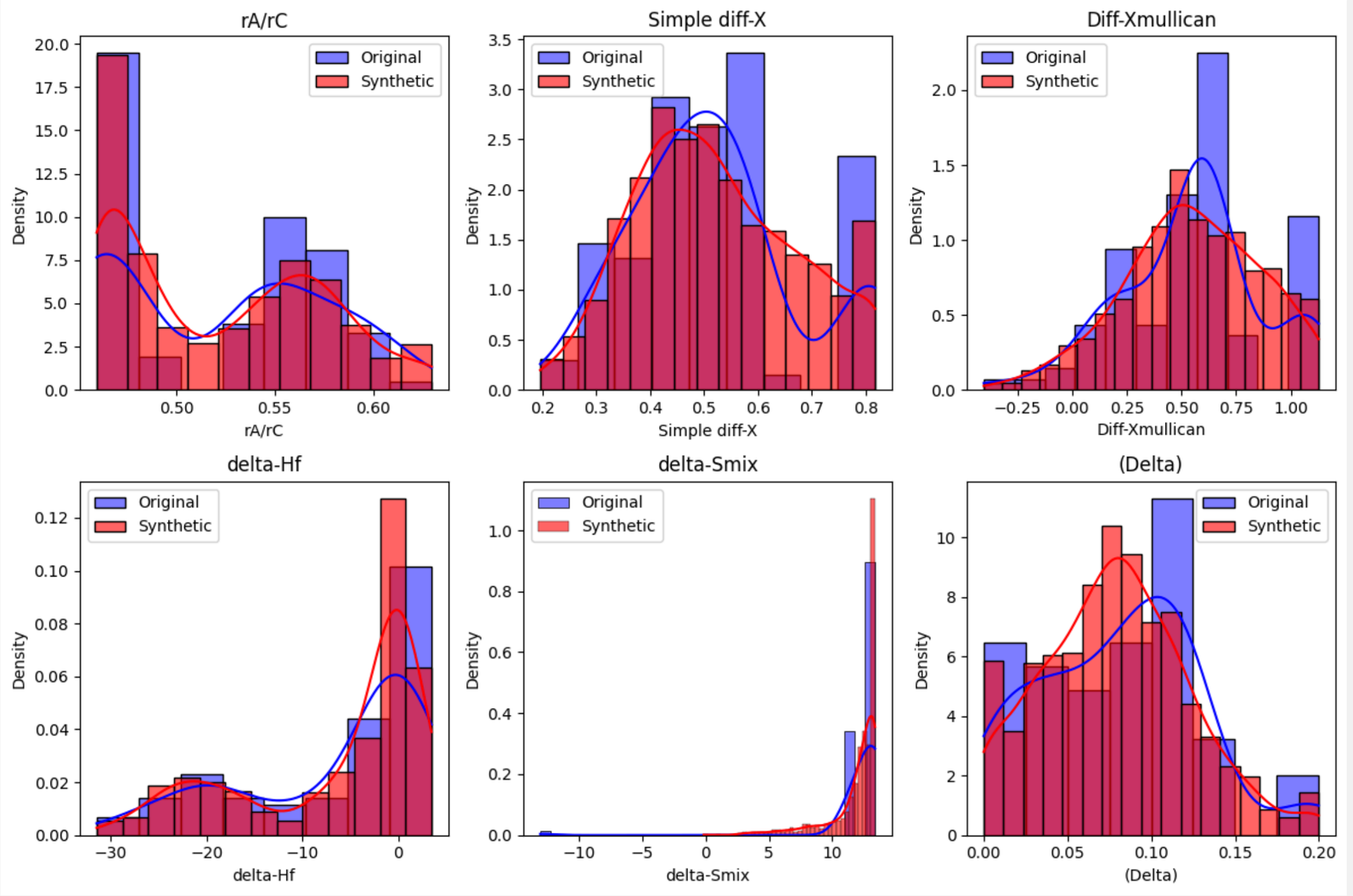
04. Using Bayesian Optimization techniques to find the best hyperparameters to obtain best accuracy

Synthetic Data Generation using Gaussian Mixture Models

Issues:

Synthetic data is generating considering independent distribution for each feature, failing to consider the feature influence on phase formation.

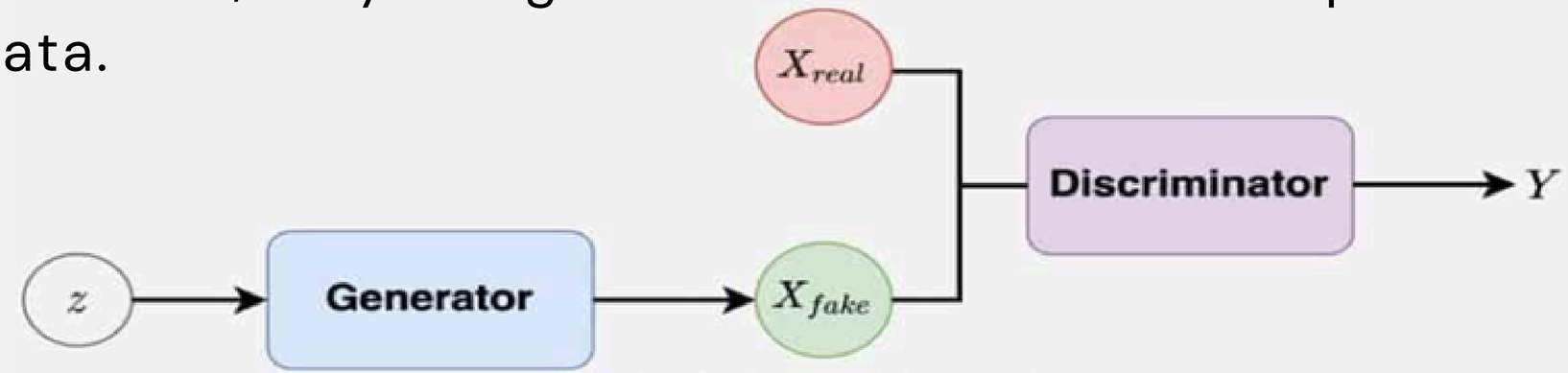
Complex process to build Multiple Mixture models considering joint probability distribution & correlations among features.



What are GANs ?

Generator → Discriminator

- GANs are composed of 2 models that compete against each other in order to produce a good distribution over the latent space.
- Generator takes some noise as input, as output generates data (similar to decoder in VAE)
- Discriminator classifies the data as real & fake. Over time each model tries to one up each other that's why they are called adversarial. It happens until the generator produces a realistic data.
- Later remove the discriminator, only the generator takes noise as input and output's realistic data.

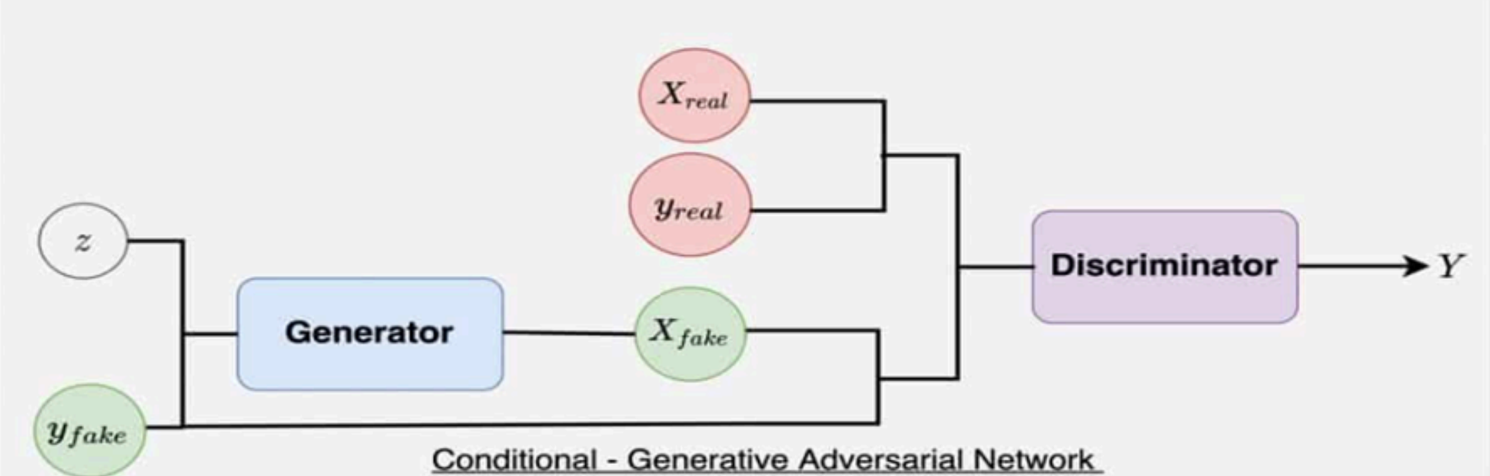


Generative Adversarial Network

Conditional Tabular GANs

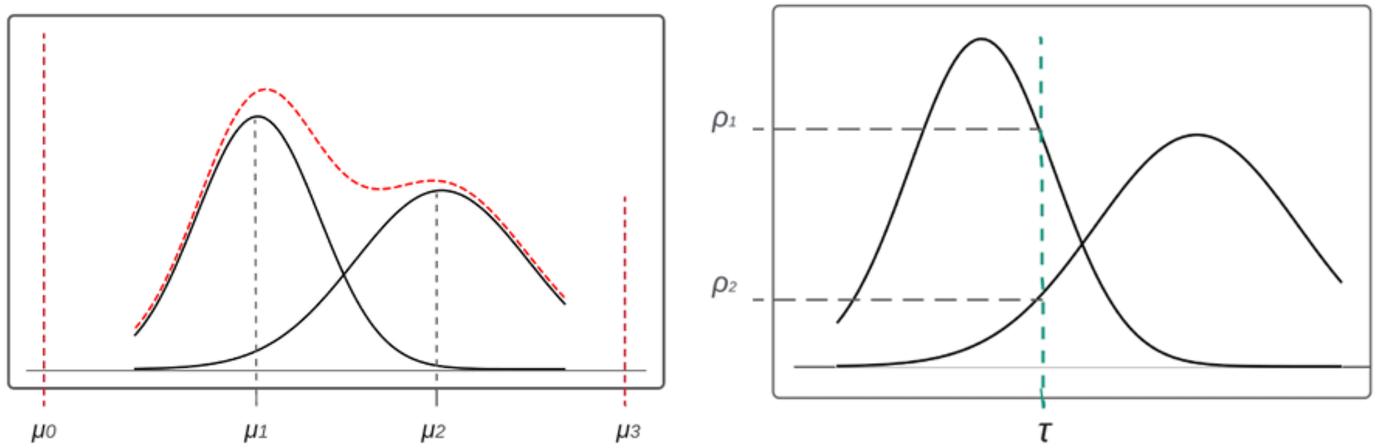
Conditional GAN Architecture: Extends traditional GANs by conditioning generation on class labels or feature values, improving control over data generation.

Numeric features: Non-Gaussian and Multimodal Distributions
CTAB-GAN introduces Mode-Specific Normalization as continuous features in tabular data are often non-Gaussian



Conditional Tabular GANs

- **Mode-specific Normalization**: Deals with multi-modal continuous features by applying clustering (e.g., Gaussian Mixture Models) and normalizing per mode, preserving statistical structure.



(a) Mixed type variable distribution with VGM (b) Mode selection of single value in continuous variable

Loss functions in CTAB-GANs

Discriminator Loss (D_loss):

$$\mathcal{L}_D = \mathbb{E}_{\tilde{x} \sim P_g}[D(\tilde{x})] - \mathbb{E}_{x \sim P_r}[D(x)] + \lambda \cdot \text{GP}$$

where GP is the gradient penalty.

- Starts high, decreases or oscillates around a stable range (e.g., -1 to 1).
- Should not collapse to zero or diverge wildly.
- GP term should stay close to 0 when gradients are behaving well.
- To reduce memorization: Add Gradient Penalty or L2 regularization.

Generator Loss (G_loss)

$$\mathcal{L}_G = -\mathbb{E}_{\tilde{x} \sim P_g}[D(\tilde{x})] + \alpha \cdot \mathcal{L}_{info}$$

- Starts high, gradually decreases.

Synthetic data generation

- Generator and discriminator losses do not go to zero or become too similar (Generator Loss ~ 0.19 and Discriminator Loss ~ -0.19)
- Loss values remain stable without extreme oscillation or sudden collapse. suggest that:
- The model has reached a sort of equilibrium—both networks are learning.

We train the model for 1000 epochs only as the discriminator and generator loss becomes quite low after these many epochs.

[3]:
```
from ctgan import CTGAN

ctgan = CTGAN(verbose=True)
ctgan.fit(data, epochs = 1000)
```

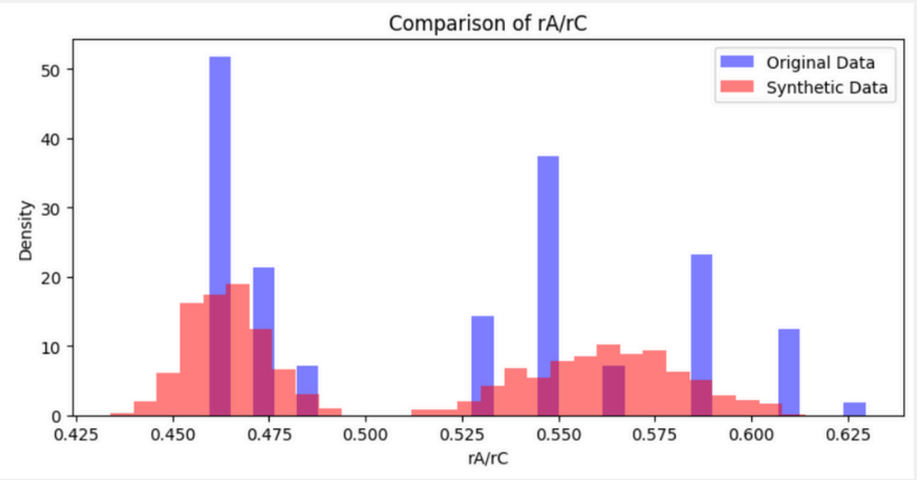
```
Gen. (0.19) | Discrim. (-0.19): 100%|████████████████████████████████████████| 1000/1000 [01:28<00:00, 11.35it/s]
```

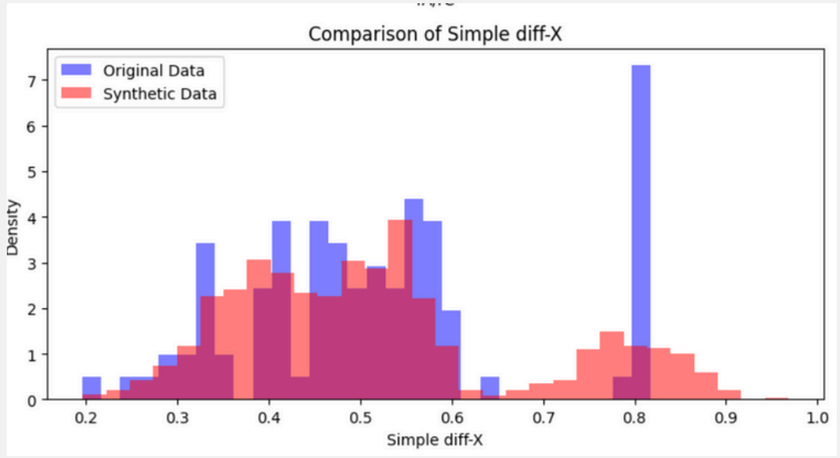
#Synthetic data generation

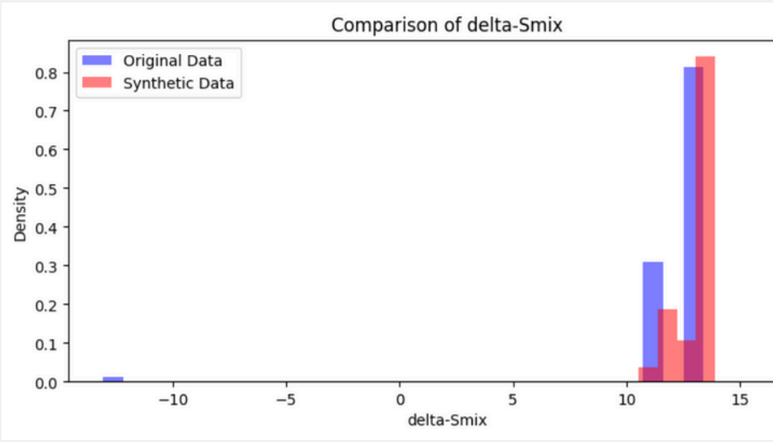
[11]:
```
samples = ctgan.sample(1000)
```

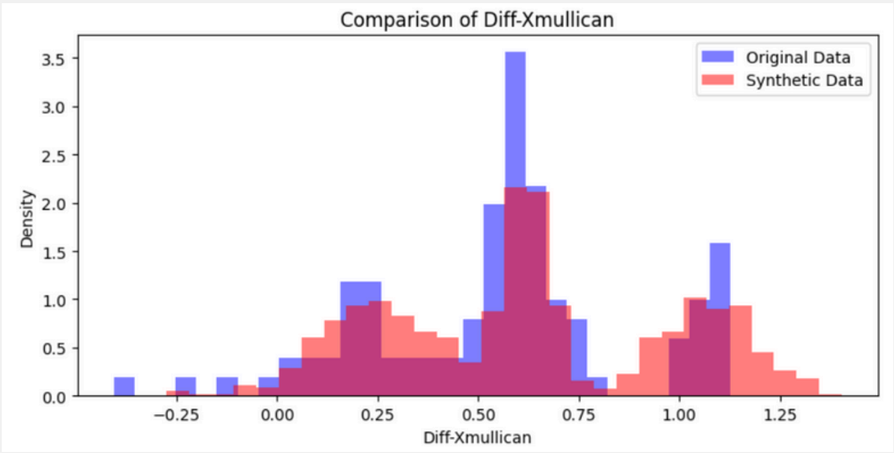
CTGAN Model **https://github.com/Team-TUD/CTAB-GAN**

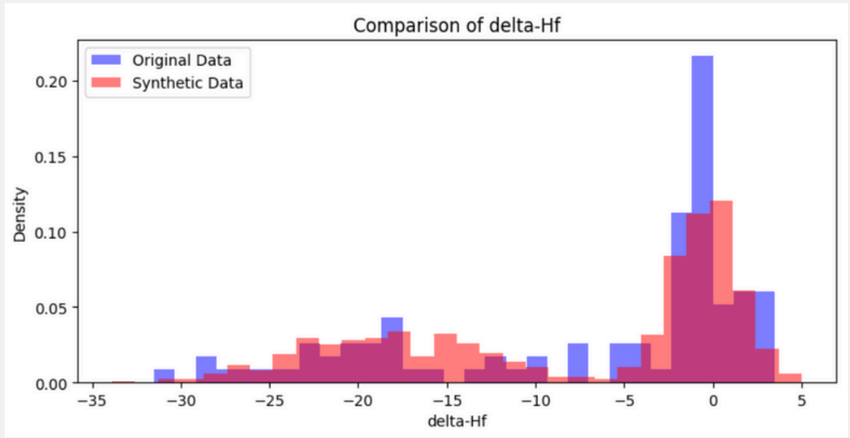
Density Plot of Real data (Blue) vs Synthetic Data (Pink)

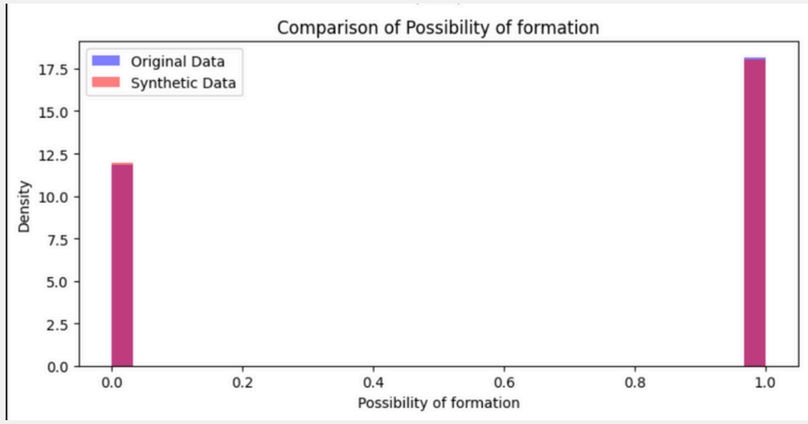












ADAYSN vs CTAB-GAN

ANDSYN:

- A statistical rule-based method.
- Constructs synthetic data using marginal distributions and some correlation structures.
- Fast, interpretable, low compute.
- But: Struggles with non-linear dependencies, multimodal distributions, and complex feature interactions.

CTAB-GAN:

- Deep learning-based, learns complex joint distributions.
- Handles mixed data types (categorical + continuous).
- Captures non-linear correlations and rare feature combinations.
- Produces more realistic and ML-useful data .

KNN ON SYNTHETIC & EXPERIMENTAL DATA WITH BAYESIAN OPTIMIZATION

KNN AND BAYESIAN OPTIMIZATION

- KNN (K-Nearest Neighbors): A classification algorithm that assigns a class based on the majority of its nearest neighbors.

- Bayesian Optimization: A method for hyperparameter optimization using a probabilistic model to find the best configuration.

DATA SCALING

```
# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

- KNN is sensitive to the scale of the data, so it’s crucial to standardize features.

- We use the StandardScaler to normalize the data to a standard scale before applying the KNN algorithm.

BAYESIAN OPTIMIZATION FOR KNN HYPERPARAMETER TUNING

Goal:

- Optimize the n_neighbors parameter of the KNN classifier.

Bayesian Optimization Approach:

- Define a function knn_evaluate(n_neighbors) to evaluate the model's performance.
- Use Bayesian Optimization to search for the best value of n_neighbors.

```python
# Define Bayesian Optimization function
def knn_evaluate(n_neighbors):
    n_neighbors = int(n_neighbors)
    knn = KNeighborsClassifier(n_neighbors=n_neighbors)
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)
    return accuracy_score(y_test, y_pred)

# Define Bayesian Optimizer
optimizer = BayesianOptimization(
    f=knn_evaluate,
    pbounds={'n_neighbors': (1, 20)},  # K values between 1 and 20
    random_state=42,
)

# Run optimization
optimizer.maximize(init_points=5, n_iter=25)
```

BAYESIAN OPTIMIZATION SETUP

Optimization Setup:

- Objective: To find the optimal value for the n_neighbors parameter.
- Parameter Bounds: Search for n_neighbors between 1 and 20.

Explanation:

- The optimizer will explore values for n_neighbors within the specified bounds to maximize accuracy.

```python
# Define Bayesian Optimization function
def knn_evaluate(n_neighbors):
    n_neighbors = int(n_neighbors)
    knn = KNeighborsClassifier(n_neighbors=n_neighbors)
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)
    return accuracy_score(y_test, y_pred)

# Define Bayesian Optimizer
optimizer = BayesianOptimization(
    f=knn_evaluate,
    pbounds={'n_neighbors': (1, 20)},  # K values between 1 and 20
    random_state=42,
)

# Run optimization
optimizer.maximize(init_points=5, n_iter=25)
```

OPTIMIZATION PROCESS

Optimizing the Model:

- The Bayesian Optimization process aims to maximize accuracy by exploring different n_neighbors values.

Explanation:

- init_points=5: The optimizer starts with 5 random points.
- n_iter=25: The optimizer then performs 25 additional iterations to refine the search.

```python
# Define Bayesian Optimization function
def knn_evaluate(n_neighbors):
    n_neighbors = int(n_neighbors)
    knn = KNeighborsClassifier(n_neighbors=n_neighbors)
    knn.fit(X_train_scaled, y_train)
    y_pred = knn.predict(X_test_scaled)
    return accuracy_score(y_test, y_pred)

# Define Bayesian Optimizer
optimizer = BayesianOptimization(
    f=knn_evaluate,
    pbounds={'n_neighbors': (1, 20)},  # K values between 1 and 20
    random_state=42,
)

# Run optimization
optimizer.maximize(init_points=5, n_iter=25)
```

BEST HYPERPARAMETER AND MODEL TRAINING

Extracting Best Parameters::

- After optimization, the best n_neighbors is found.

Model Training:

- The KNN classifier is trained using the best n_neighbors value.

Explanation:

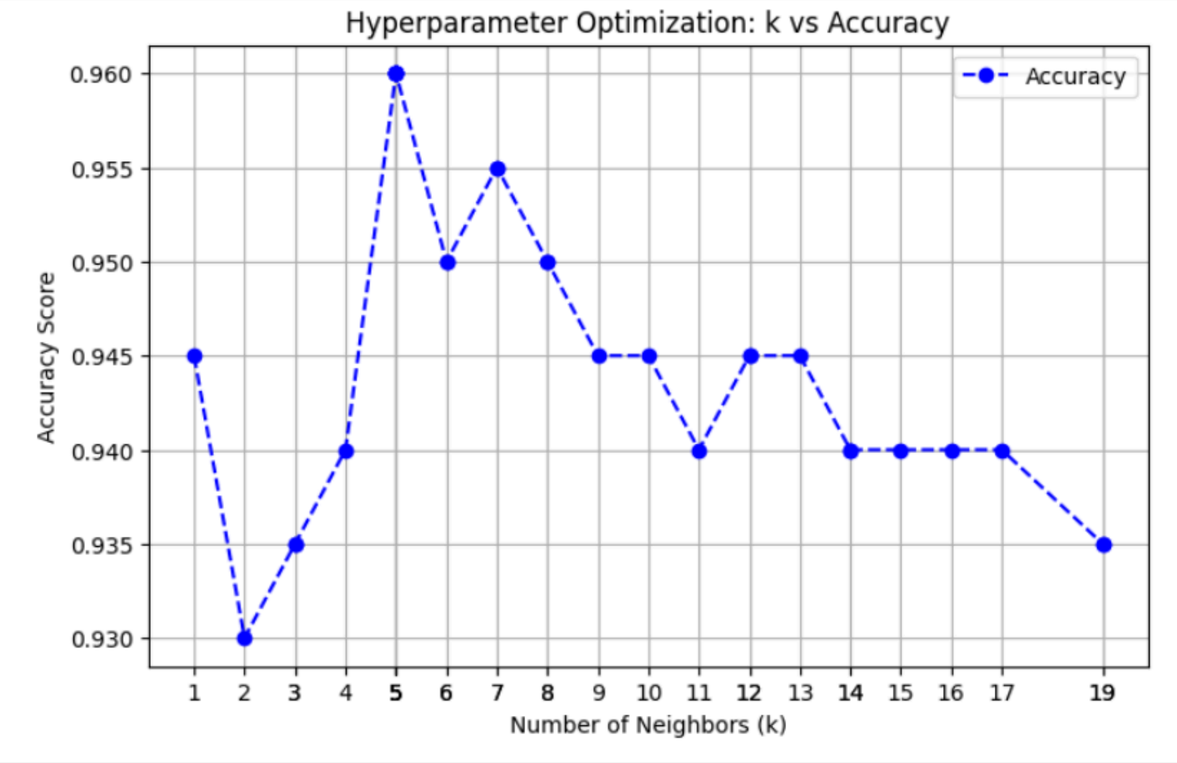
- The model is now trained with the optimized hyperparameter, ready for evaluation.

```
# Get best parameters
best_k = int(optimizer.max['params']['n_neighbors'])
print(f"Best number of neighbors: {best_k}")

# Train optimized KNN model
knn_model = KNeighborsClassifier(n_neighbors=best_k)
knn_model.fit(X_train_scaled, y_train)
```

K- VALUE VS ACCURACY

(For synthetic data)



On synthetic data

Explanation:

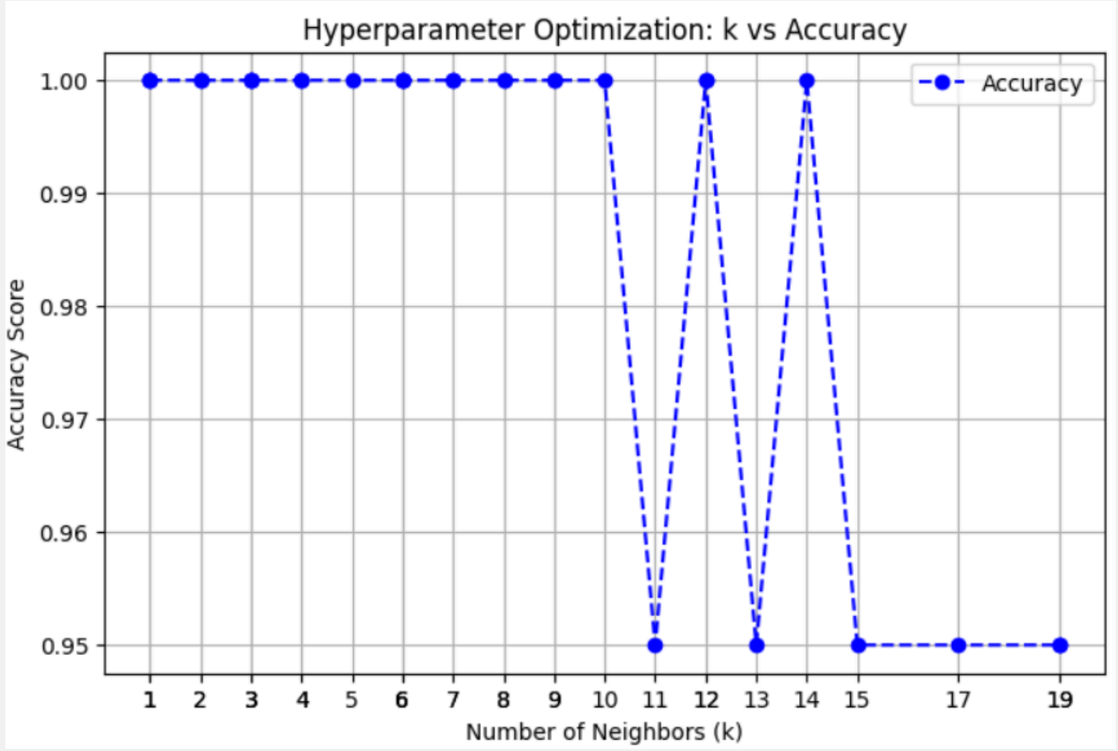
- The plot shows how the model's accuracy changes as the value of k (number of neighbors) increases.
- X-axis: Number of Neighbors (k).
- Y-axis: Accuracy Score.
- The plot uses dashed blue lines with dots representing the accuracy for each k value.

Insight:

- The best accuracy is achieved when k=5 with an accuracy of 0.960
- Stable accuracy is achieved for K>=9 with an accuracy of 0.945

K- VALUE VS ACCURACY

(For Training data)



On Training data

Explanation:

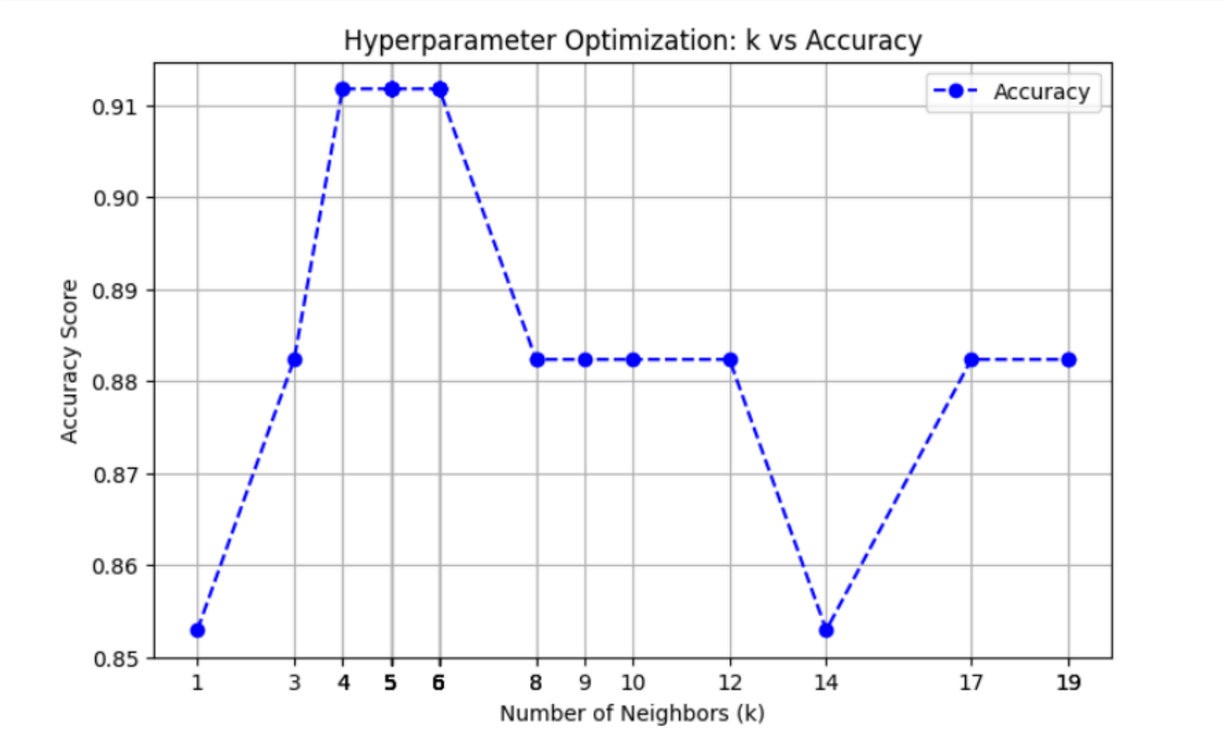
- The plot shows how the model's accuracy changes as the value of k (number of neighbors) increases.
- X-axis: Number of Neighbors (k).
- Y-axis: Accuracy Score.
- The plot uses dashed blue lines with dots representing the accuracy for each k value.

Insight:

- The best accuracy is achieved when k>=1 with an accuracy of 0.999
- Stable accuracy is achieved for K>=1 with an accuracy of 0.999

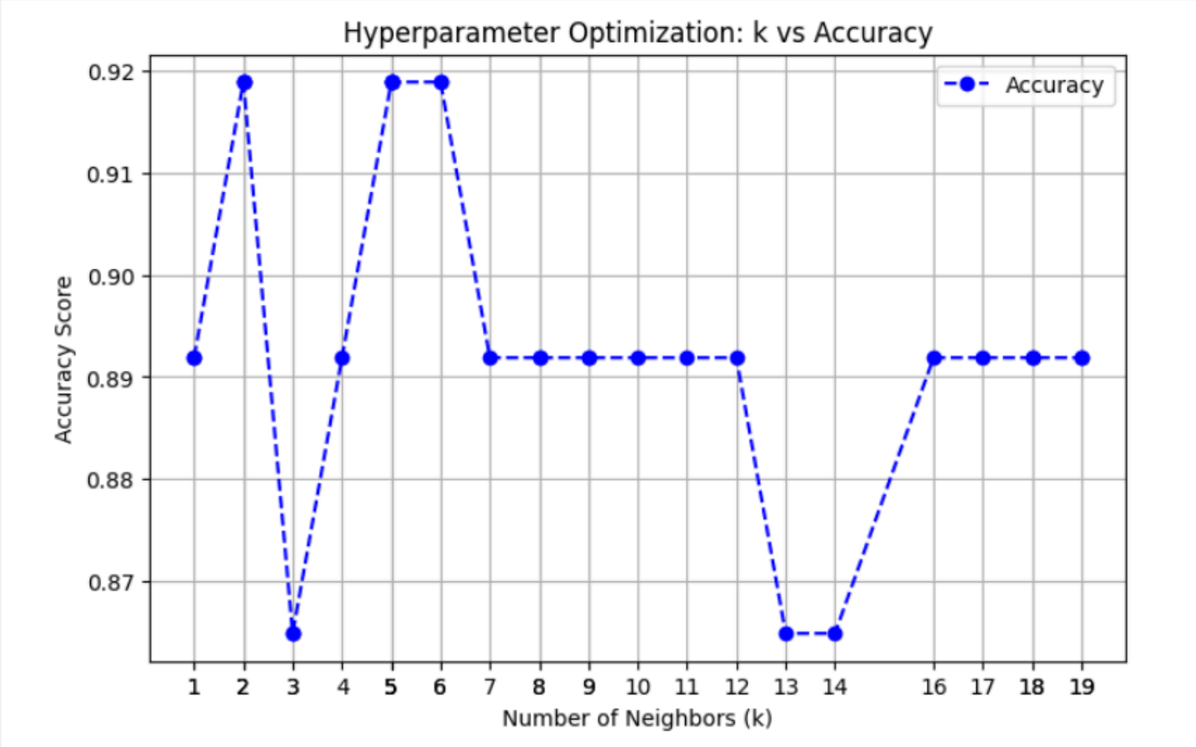
K- VALUE VS ACCURACY

(For synthetic data)



On testing data

- The best accuracy is achieved when k=4 with an accuracy of 0.912
- Stable accuracy is achieved for K>=9 with an accuracy of 0.8824

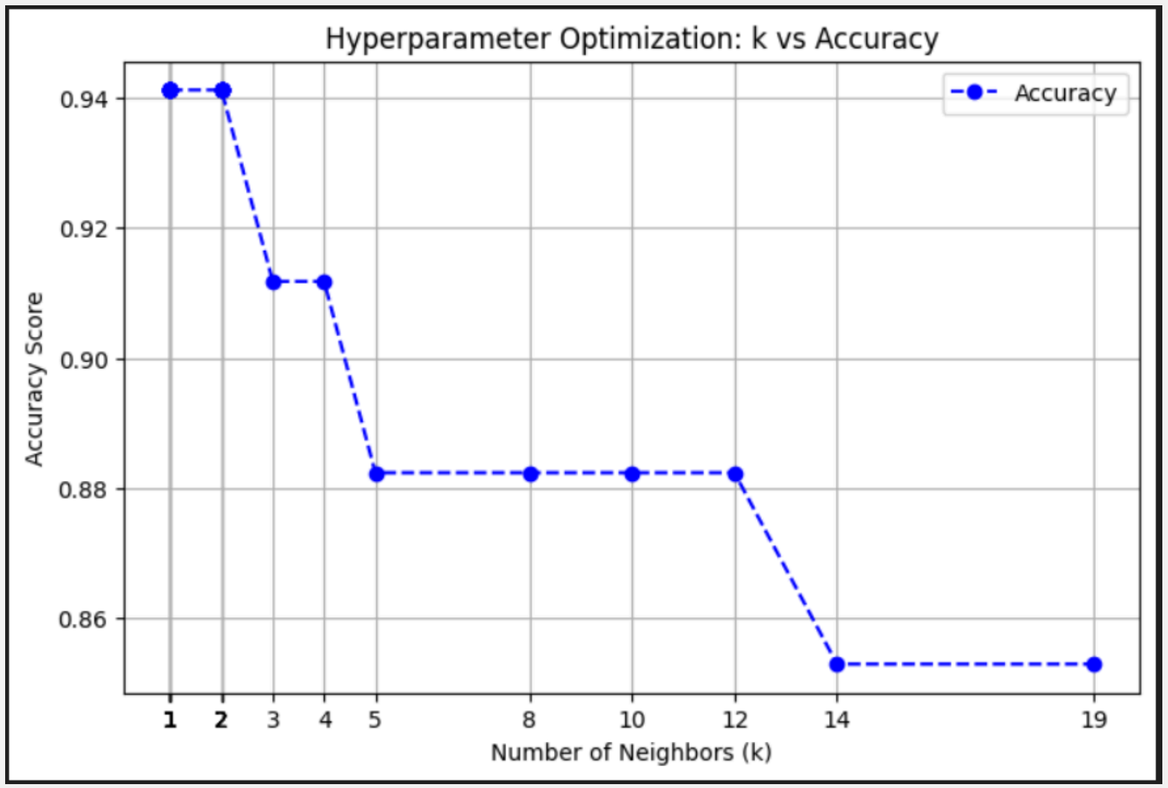


On Suppliment data

- The best accuracy is achieved when k=5 with an accuracy of 0.92
- Stable accuracy is achieved for K>=8 with an accuracy of 0.8919

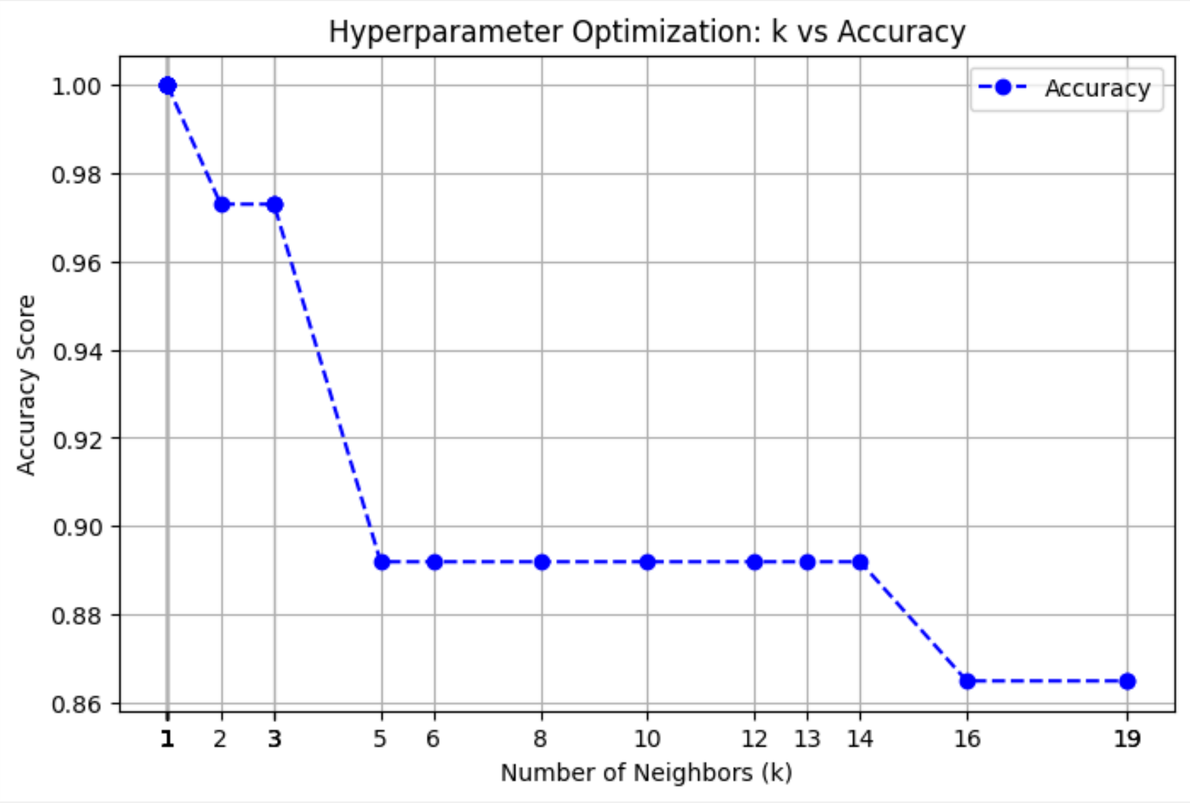
K- VALUE VS ACCURACY

(For Training data)



On testing data

- The best accuracy is achieved when k=2 with an accuracy of 0.9412
- Stable accuracy is achieved for K>=5 with an accuracy of 0.8824

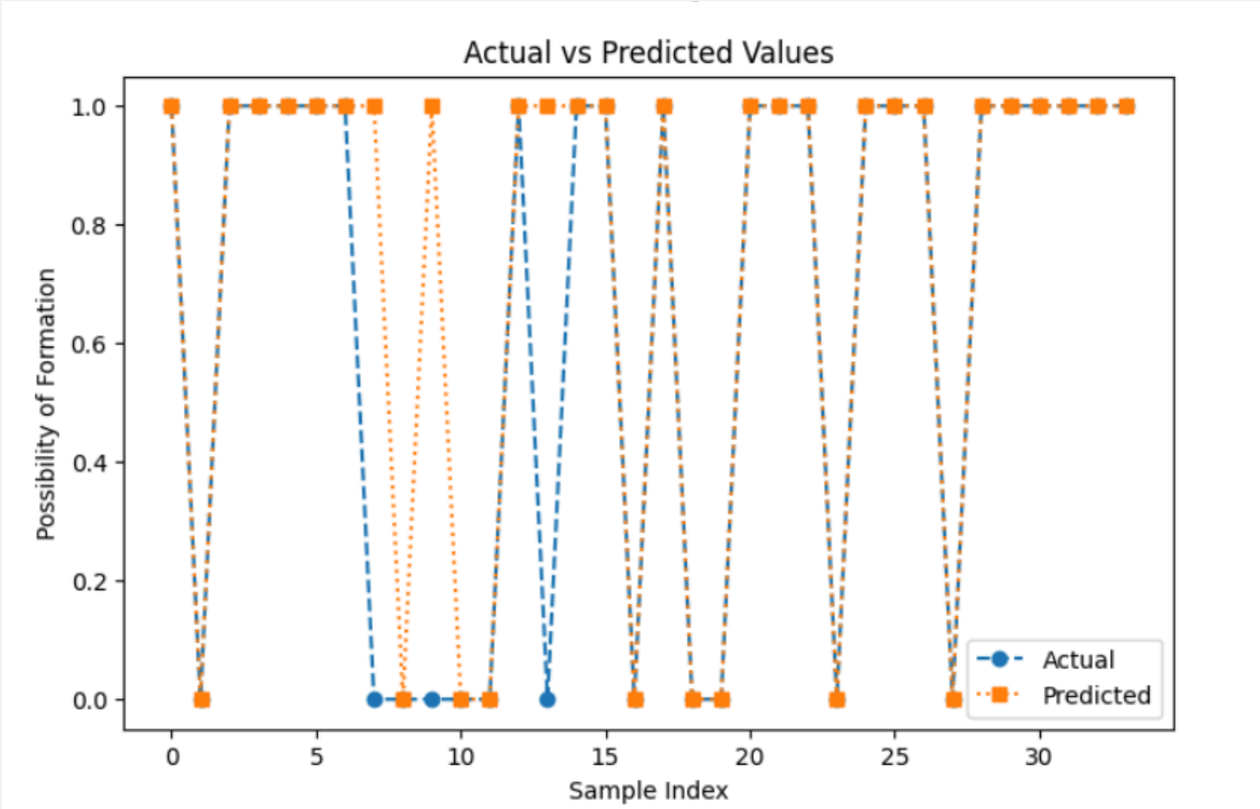


On Suppliment data

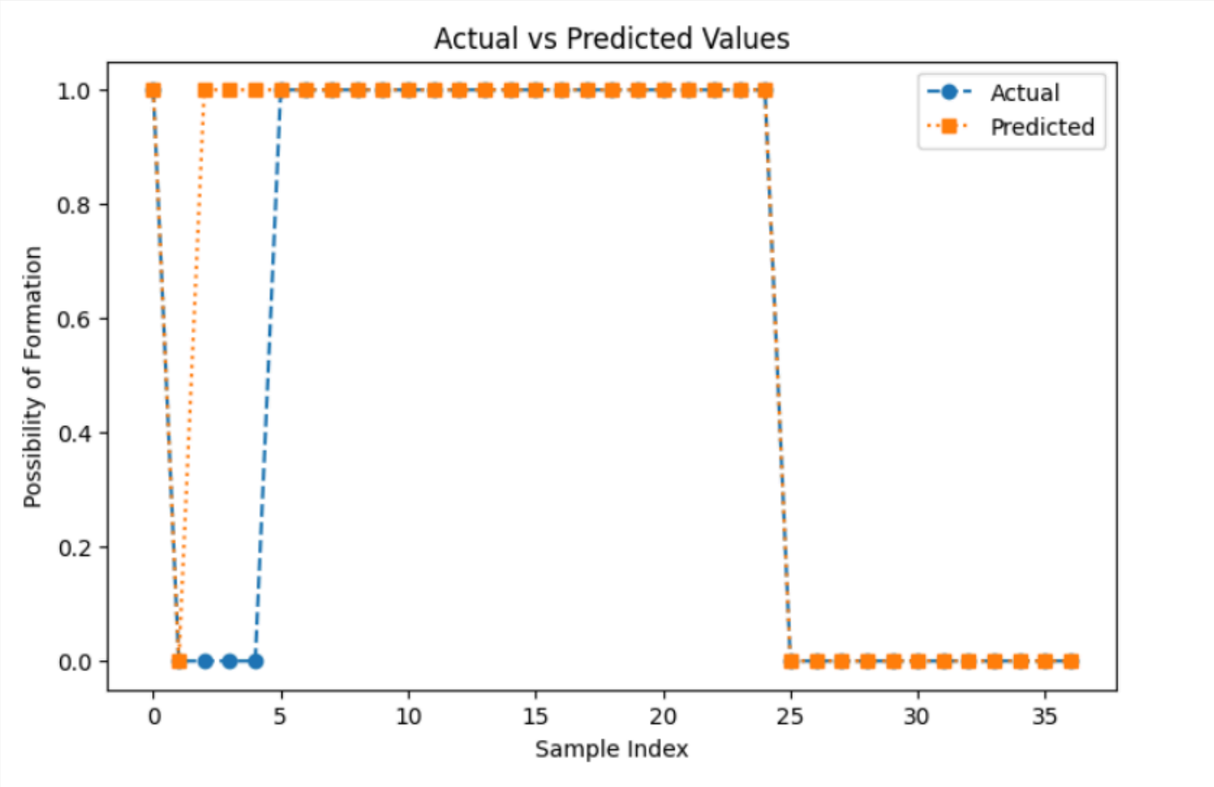
- The best accuracy is achieved when k=5 with an accuracy of 0.973
- Stable accuracy is achieved for K>=8 with an accuracy of 0.8919

ACTUAL VS PREDICTED

(For Synthetic data)



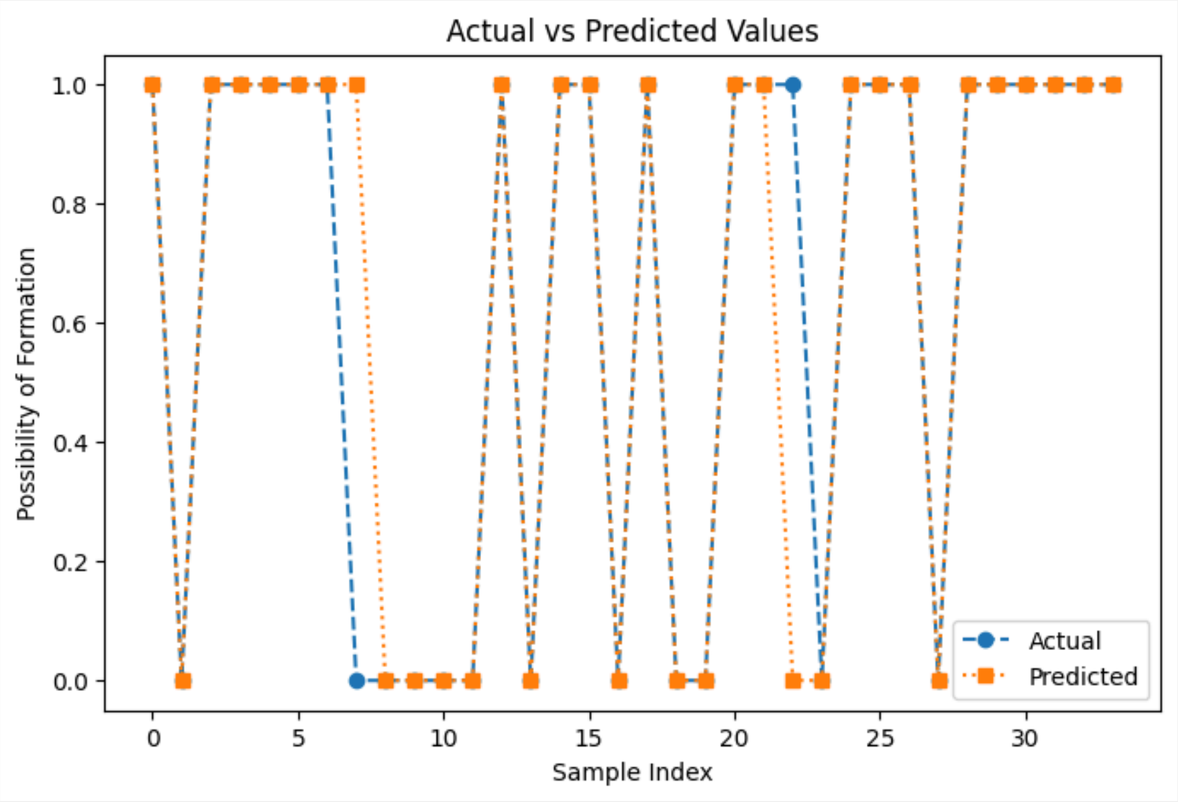
On testing data



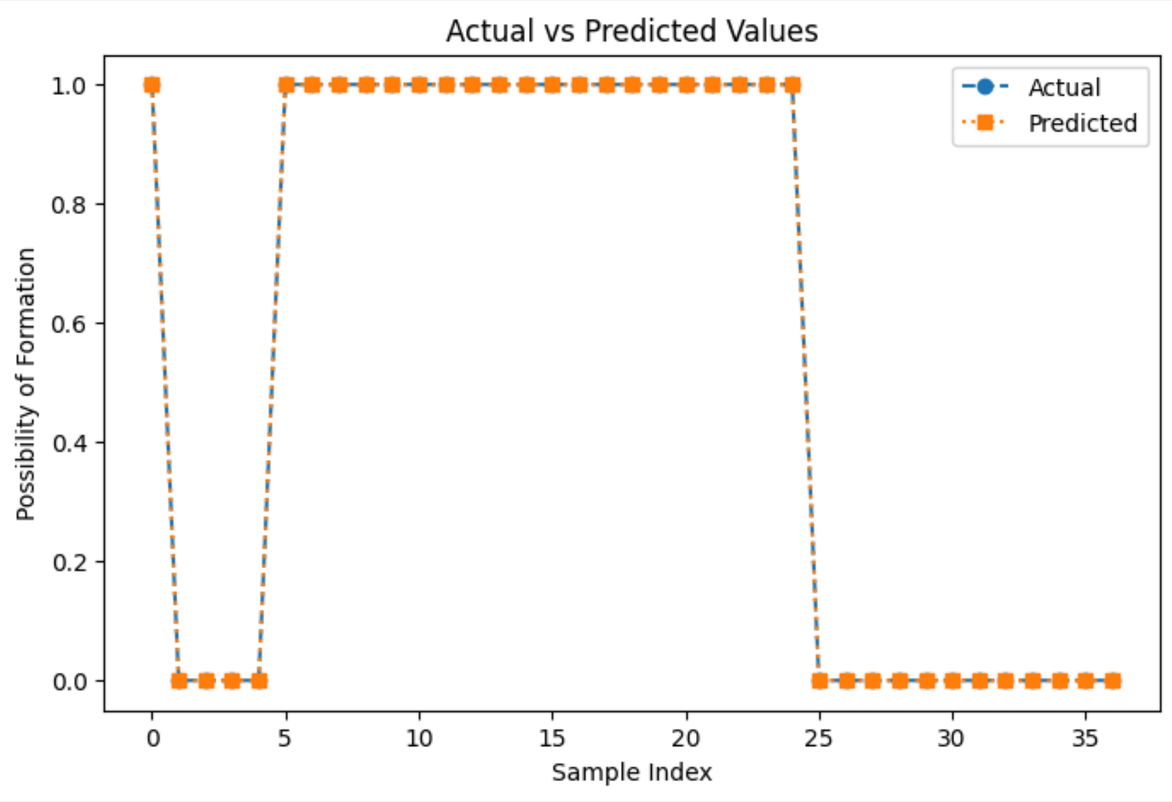
On Suppliment data

ACTUAL VS PREDICTED

(For Training data)



On testing data



On Suppliment data

RANDOM FOREST ON SYNTHETIC & EXPERIMENTAL DATA WITH BAYESIAN OPTIMIZATION

DATA PREPROCESSING AND FEATURE STANDARDIZATION

Objective:

- Prepare data for Random Forest model training by scaling features

Explanation:

- **StandardScaler**: Normalizes the feature set so that each feature has a mean of 0 and standard deviation of 1. This is crucial for models like Random Forests, especially when hyperparameters are being optimized.

```
# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

BAYESIAN OPTIMIZATION FOR RANDOM FOREST

```python
# Define Bayesian Optimization function
def rf_evaluate(n_estimators):
    n_estimators = int(n_estimators)  # Convert float to int
    rf = RandomForestClassifier(n_estimators=n_estimators, random_state=42)
    rf.fit(X_train_scaled, y_train)
    y_pred = rf.predict(X_test_scaled)
    return accuracy_score(y_test, y_pred)
```

```python
# Define Bayesian Optimizer
optimizer = BayesianOptimization(
    f=rf_evaluate,
    pbounds={'n_estimators': (10, 200)},  #
    random_state=42,
)

# Run optimization
optimizer.maximize(init_points=5, n_iter=20)
```

Objective:

- Use Bayesian Optimization to tune the hyperparameter n_estimators (number of trees) of the the Random Forest model.

Explanation:

- **Bayesian Optimization:** Efficiently searches the space for the best number of trees (estimators) in the Random Forest.
- The best n_estimators is found by maximizing accuracy over a range from 10 to 200 trees.

BEST PARAMETER AND MODEL TRAINING

Extracting Best Parameters::

- After optimization, the best n_estimator(no of trees) is found.

Model Training:

- The random Forest classifier is trained using the best n_estimator value.

Explanation:

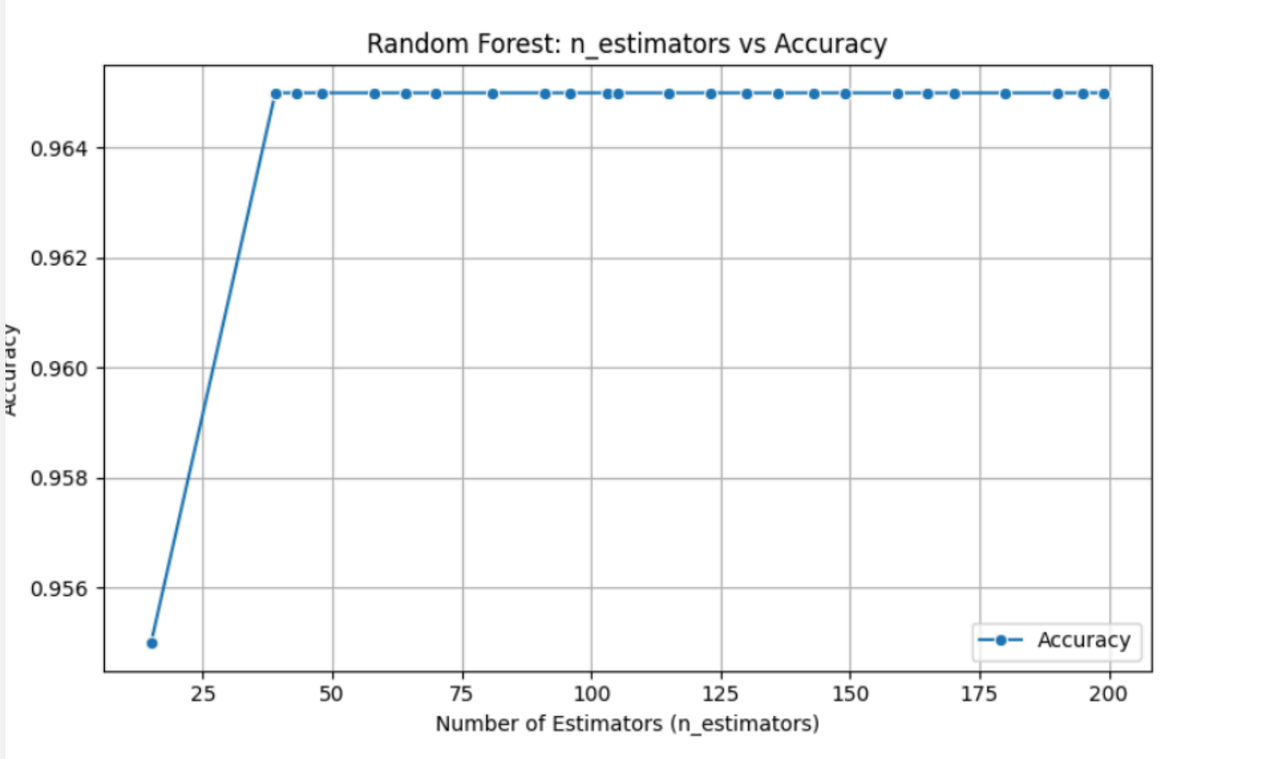
- The model is now trained with the optimized hyperparameter, ready for evaluation.

```
# Get best parameters
best_n_estimators = int(optimizer.max['params']['n_estimators'])
print(f"Best number of estimators: {best_n_estimators}")

# Train optimized Random Forest model
rf_model = RandomForestClassifier(n_estimators=best_n_estimators, random_state=42)
rf_model.fit(X_train_scaled, y_train)
```

N_ESTIMATOR VS ACCURACY

(For Synthetic data)



On Synthetic data

Explanation:

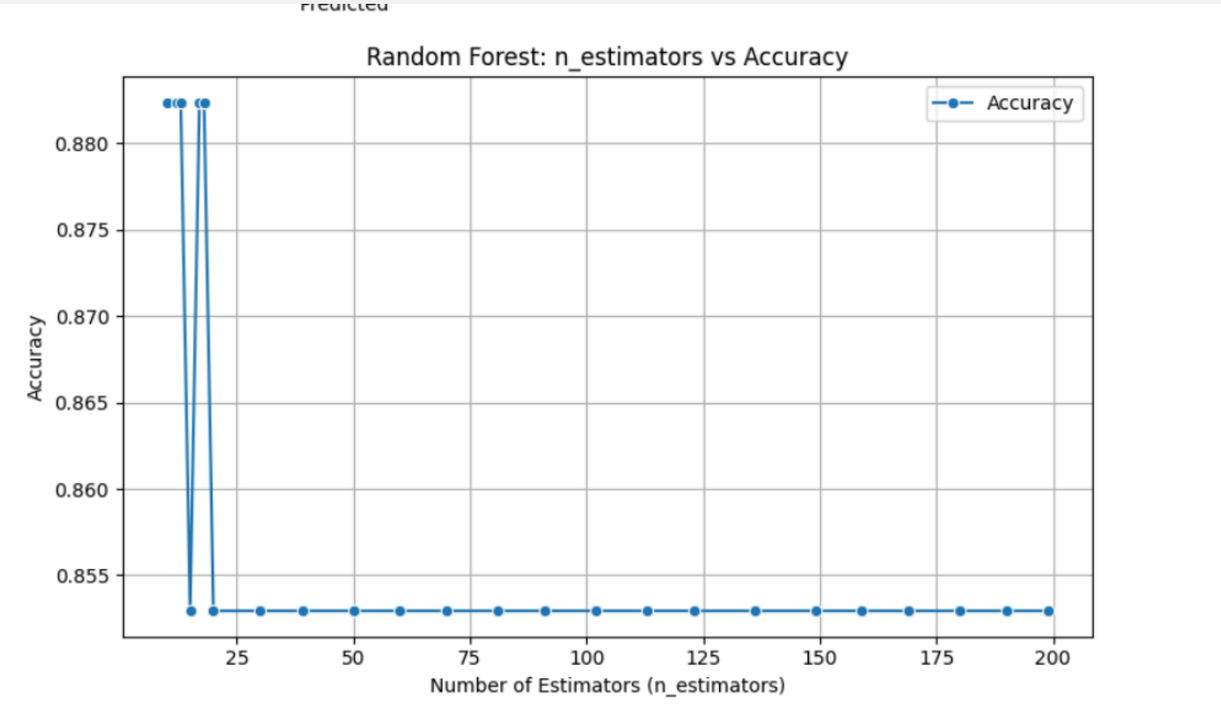
- The plot illustrates how the model's accuracy varies as the number of estimators (trees) in the Random Forest algorithm changes.
- X-axis: Number of Estimators (n_estimators), which represents the number of trees used in the Random Forest model.
- Y-axis: Accuracy Score, which shows the performance of the model as a percentage.
- The plot uses a line with blue dots representing the accuracy for each number of estimators.

Insight:

- The best accuracy is achieved when n_estimator=81 with an accuracy of 0.966
- Stable accuracy is achieved for n_estimator>=37 with an accuracy of 0.95

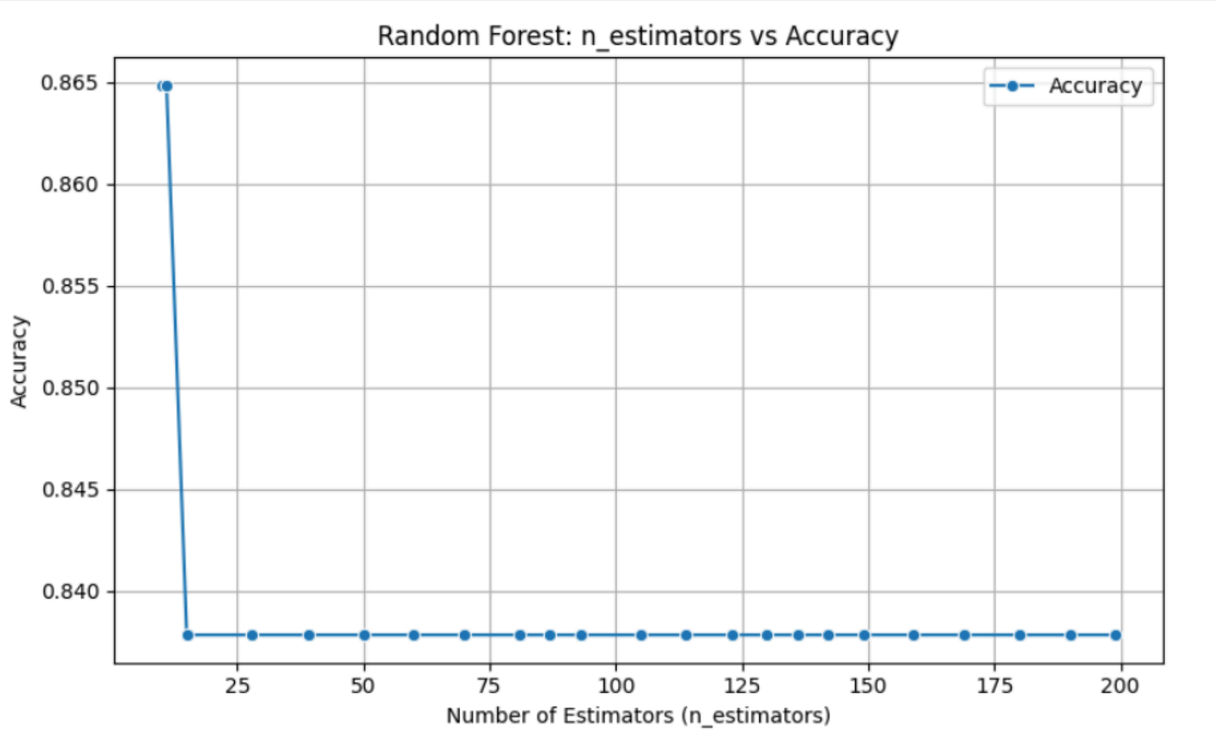
N_ESTIMATOR VS ACCURACY

(For Synthetic data)



On testing data

- The best accuracy is achieved when n_estimator=10 with an accuracy of 0.88
- Stable accuracy is achieved for n_estimator>=25 with an accuracy of 0.8529

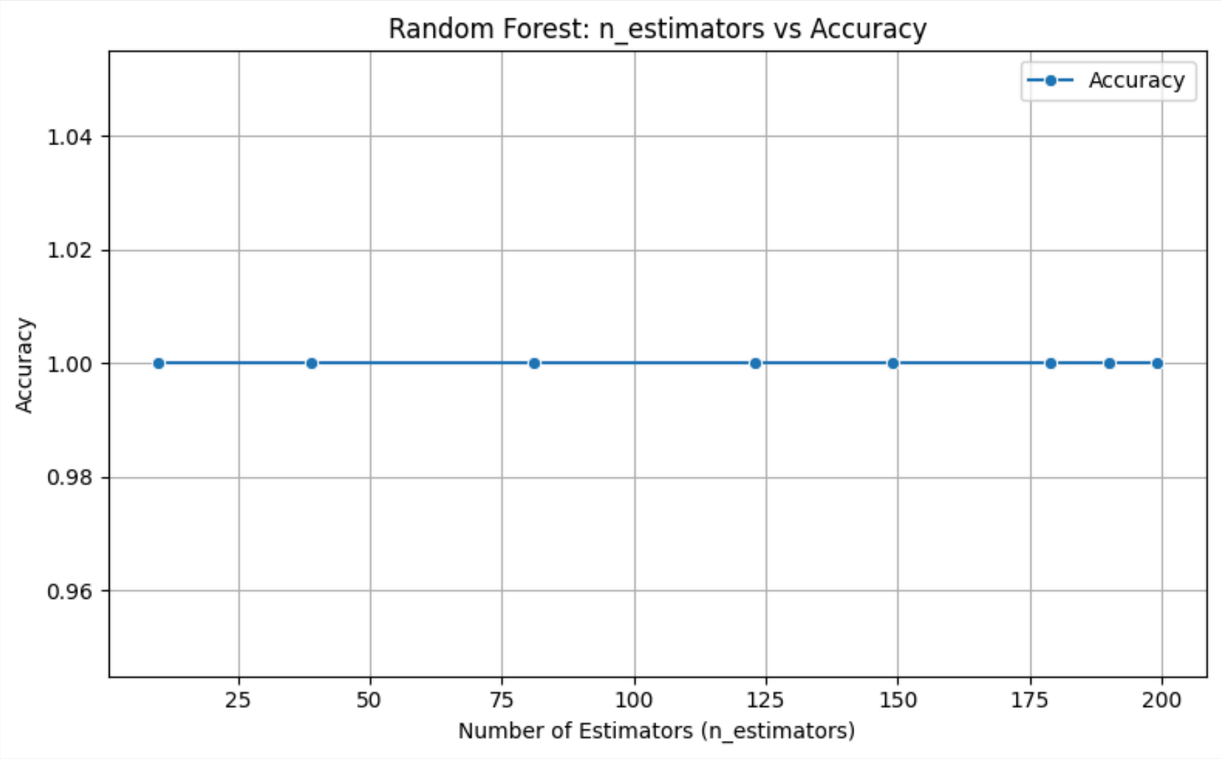


On Suppliment data

- The best accuracy is achieved when n_estimator=10 with an accuracy of 0.86
- Stable accuracy is achieved for n_estimator>=15 with an accuracy of 0.8378

N_ESTIMATOR VS ACCURACY

(For Training data)



On Training data

Explanation:

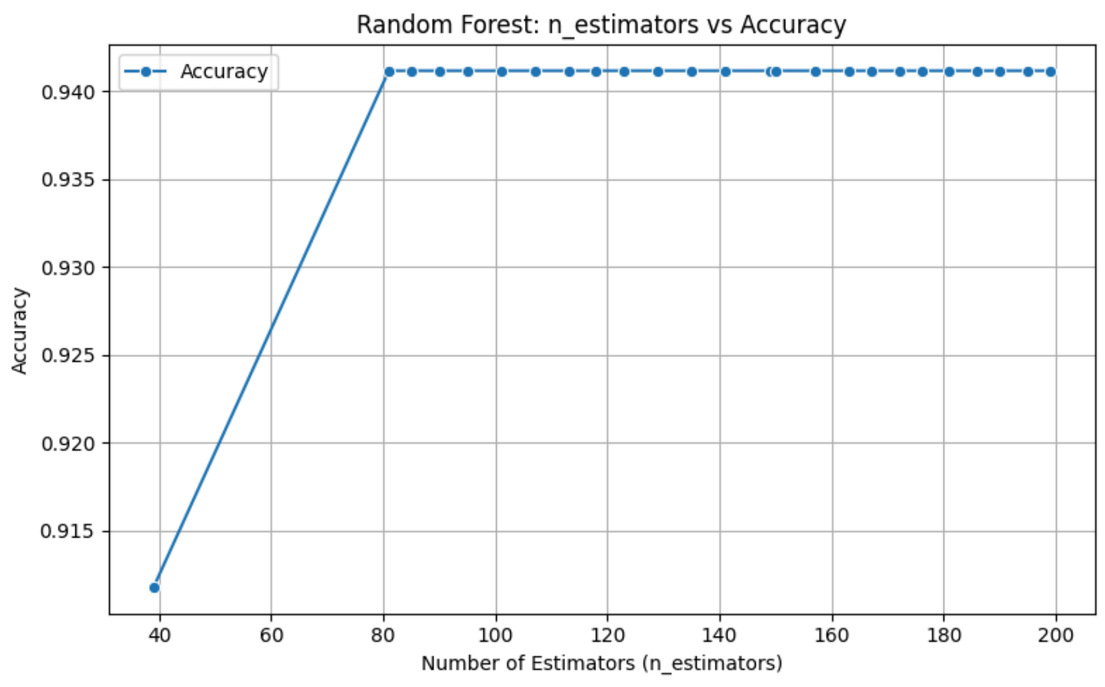
- The plot illustrates how the model's accuracy varies as the number of estimators (trees) in the Random Forest algorithm changes.
- X-axis: Number of Estimators (n_estimators), which represents the number of trees used in the Random Forest model.
- Y-axis: Accuracy Score, which shows the performance of the model as a percentage.
- The plot uses a line with blue dots representing the accuracy for each number of estimators.

Insight:

- The best accuracy is achieved with an accuracy tend to 1
- Stable accuracy is achieved with an accuracy tend 1

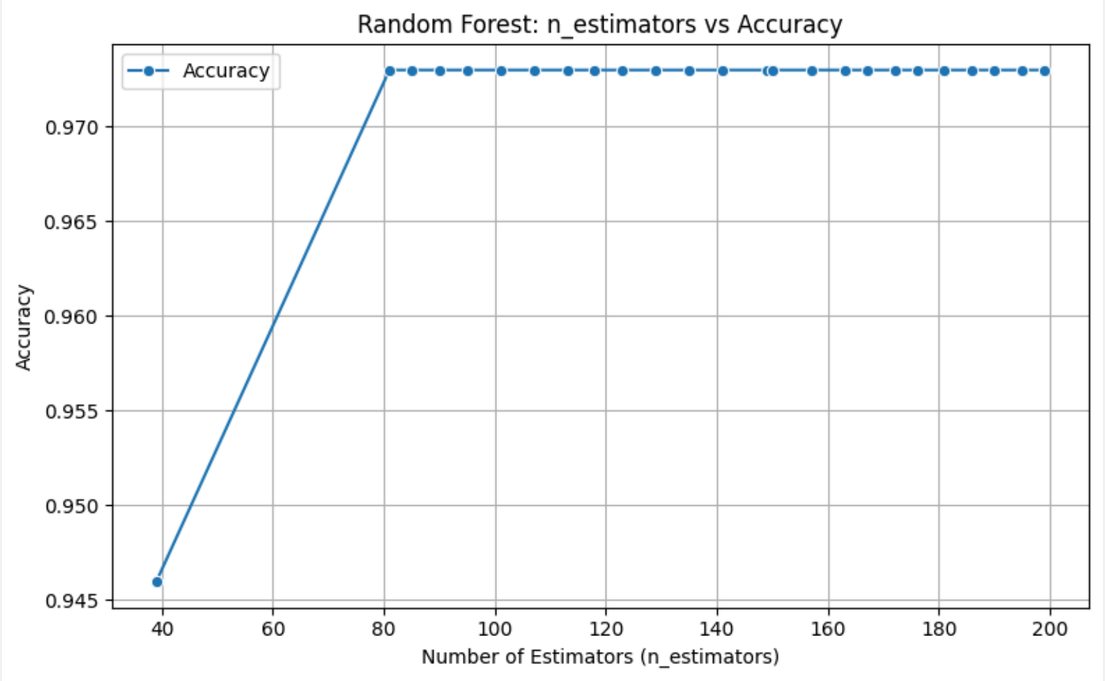
N_ESTIMATOR VS ACCURACY

(For Training data)



On testing data

- The best accuracy is achieved when n_estimator>=80 with an accuracy of 0.9412
- Stable accuracy is achieved for n_estimator>=80 with an accuracy of 0.9412

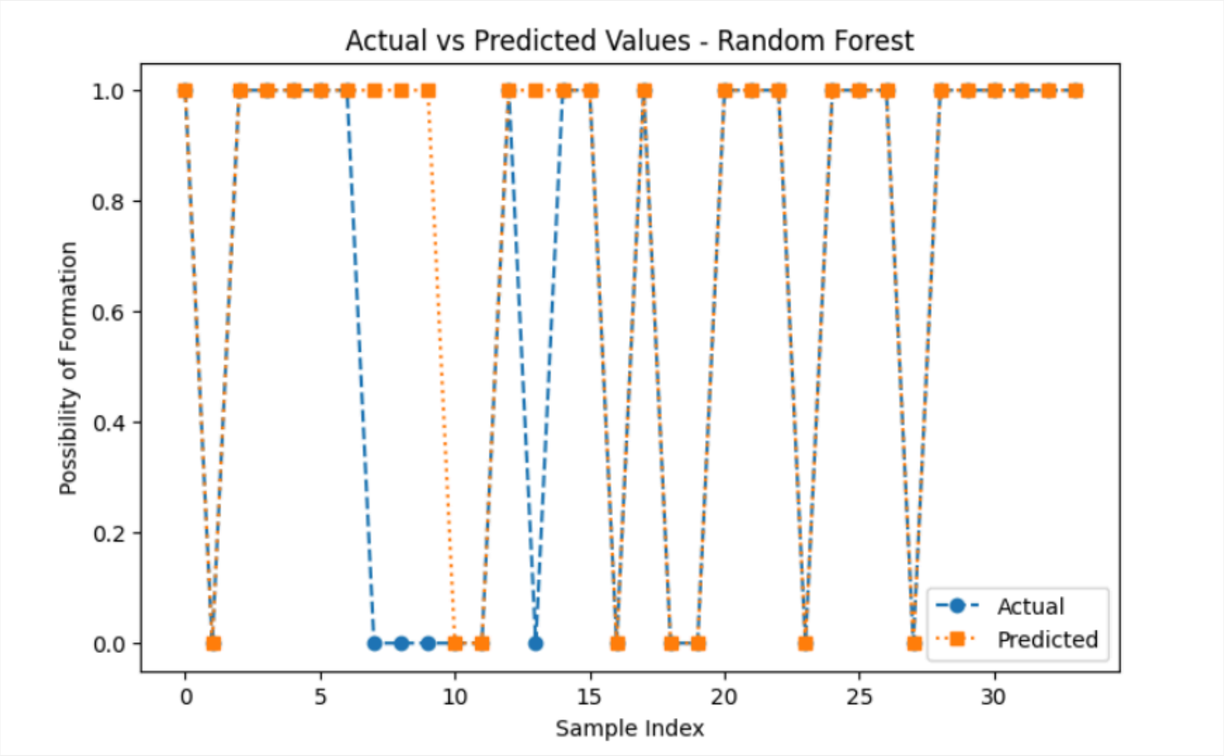


On Suppliment data

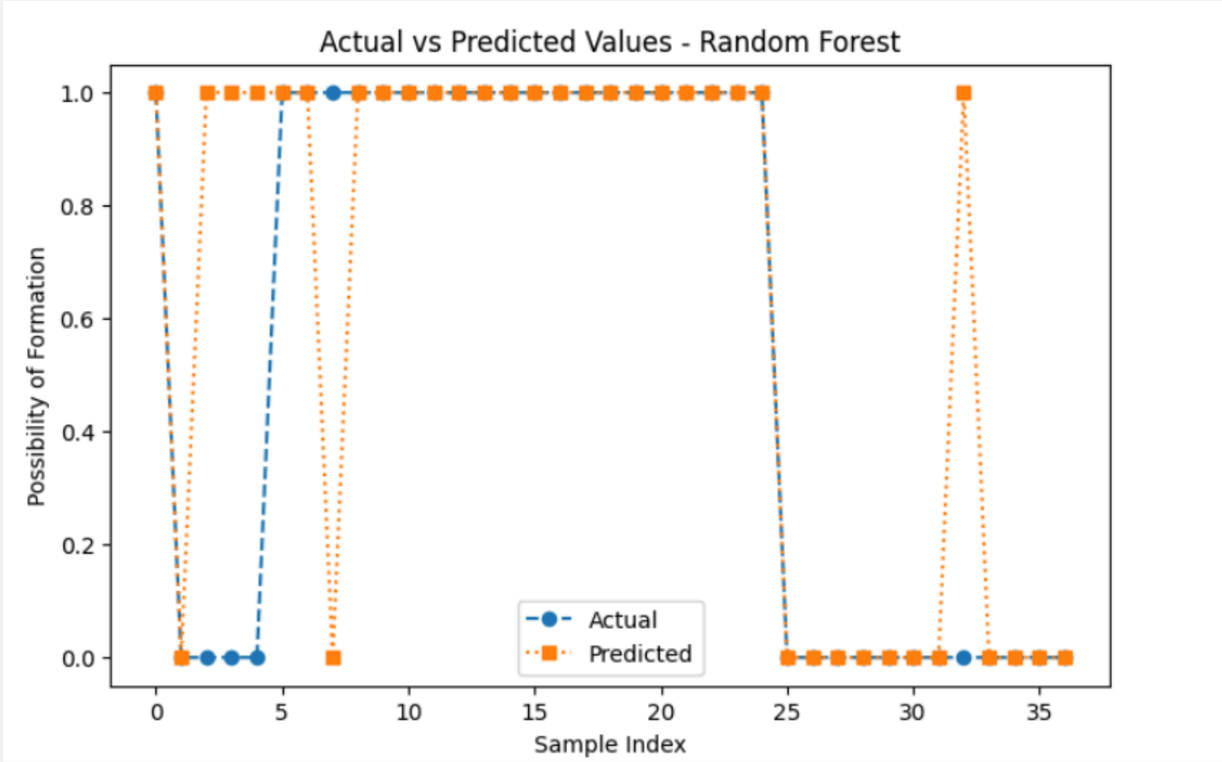
- The best accuracy is achieved when n_estimator>=80 with an accuracy of 0.973
- Stable accuracy is achieved for n_estimator>=80 with an accuracy of 0.973

ACTUAL VS PREDICTED

(For Synthetic data)



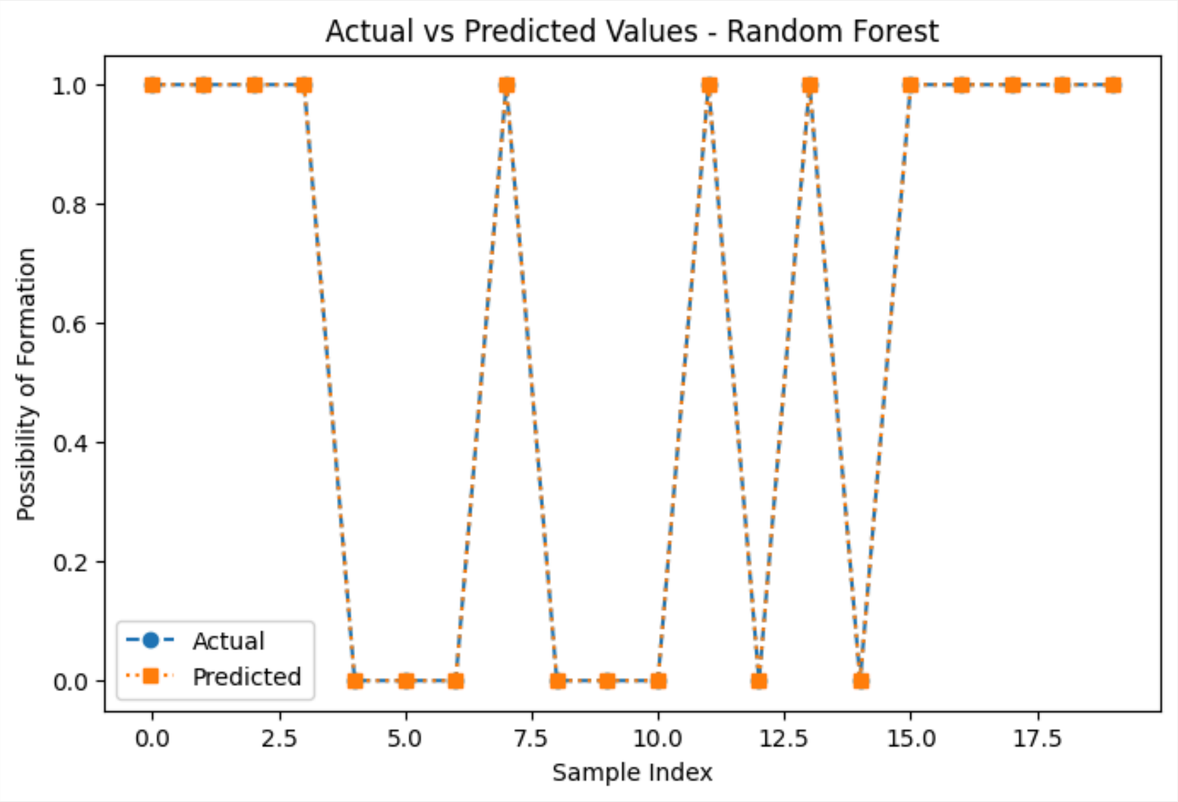
On testing data



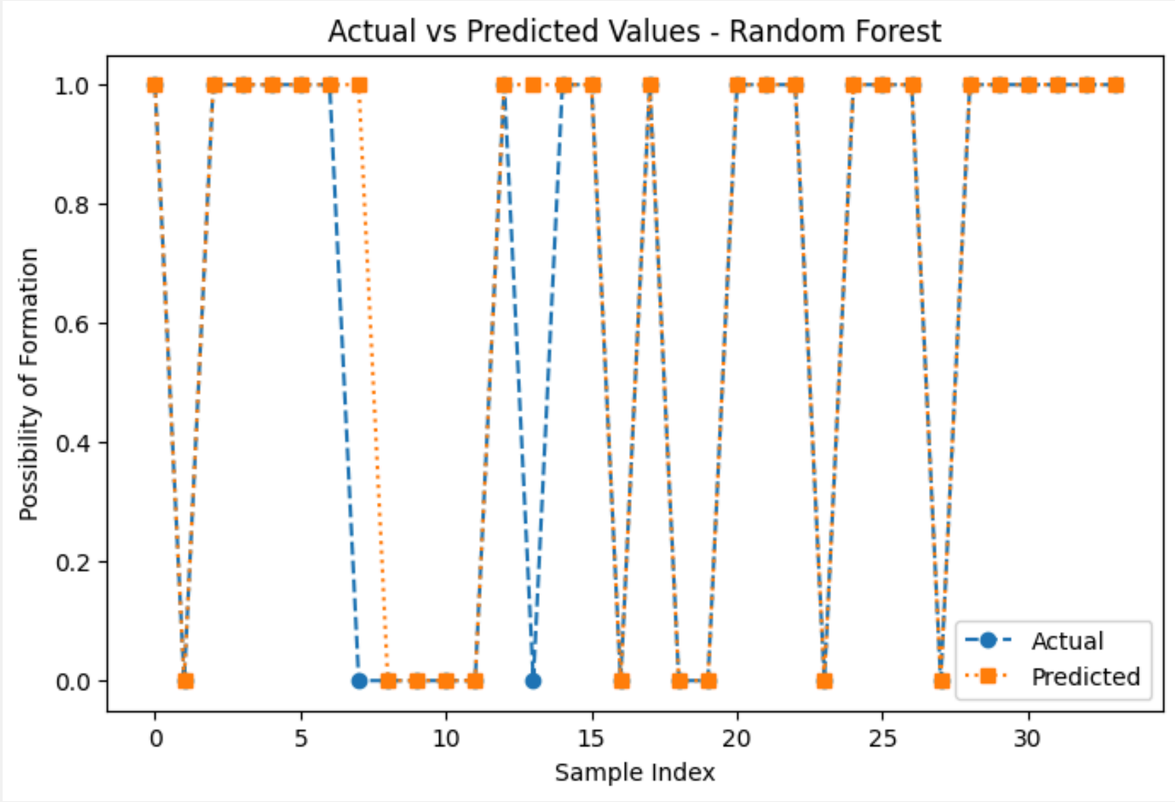
On Suppliment data

ACTUAL VS PREDICTED

(For Training data)



On testing data



On Suppliment data

SVM ON SYNTHETIC & EXPERIMENTAL DATA WITH BAYESIAN OPTIMIZATION

DATA STANDARDIZATION AND EVALUATION FUNCTION

```python
# --- Standardize Features ---
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X)
X_test_scaled = scaler.transform(X_test)
```

```python
# --- Define Evaluation Function ---
def svm_evaluate(C, gamma):
    model = SVC(C=10**C, gamma=10**gamma, kernel='rbf', random_state=42)
    model.fit(X_train_scaled, y)
    y_pred = model.predict(X_test_scaled)
    return accuracy_score(y_test, y_pred)
```

Explanation:

- To ensure all features are on the same scale, the data is standardized using StandardScaler from sklearn. This step improves the performance of many machine learning algorithms, including SVMs.
- The evaluation function (svm_evaluate) defines an SVM model, fits it to the scaled training data, makes predictions, and computes the accuracy score. This function is the target for Bayesian optimization.

BAYESIAN OPTIMIZATION SETUP

```
# --- Set Bounds in log-scale for numerical stability ---
pbounds = {
    'C': (-3, 3),         # 10^-3 to 10^3
    'gamma': (-4, 1)      # 10^-4 to 10^1
}

# --- Run Bayesian Optimization ---
optimizer = BayesianOptimization(
    f=svm_evaluate,
    pbounds=pbounds,
    random_state=42
)

optimizer.maximize(init_points=5, n_iter=15)
```

Explanation:

- Bayesian Optimization is used to search for the best hyperparameters (C and gamma) by maximizing the evaluation function (svm_evaluate). The pbounds dictionary defines the bounds for each hyperparameter in log scale.

BEST HYPERPARAMETERS AND FINAL MODEL TRAINING

```python
# --- Best Parameters ---
best_params = optimizer.max['params']
best_C = 10**best_params['C']
best_gamma = 10**best_params['gamma']
print(f"\nBest SVM Parameters: C={best_C:.4f}, gamma={best_gamma:.4f}")
```

```python
# --- Train Final Model with Best Params ---
best_svm = SVC(C=best_C, gamma=best_gamma, kernel='rbf', random_state=42)
best_svm.fit(X_train_scaled, y)
y_pred = best_svm.predict(X_test_scaled)
```

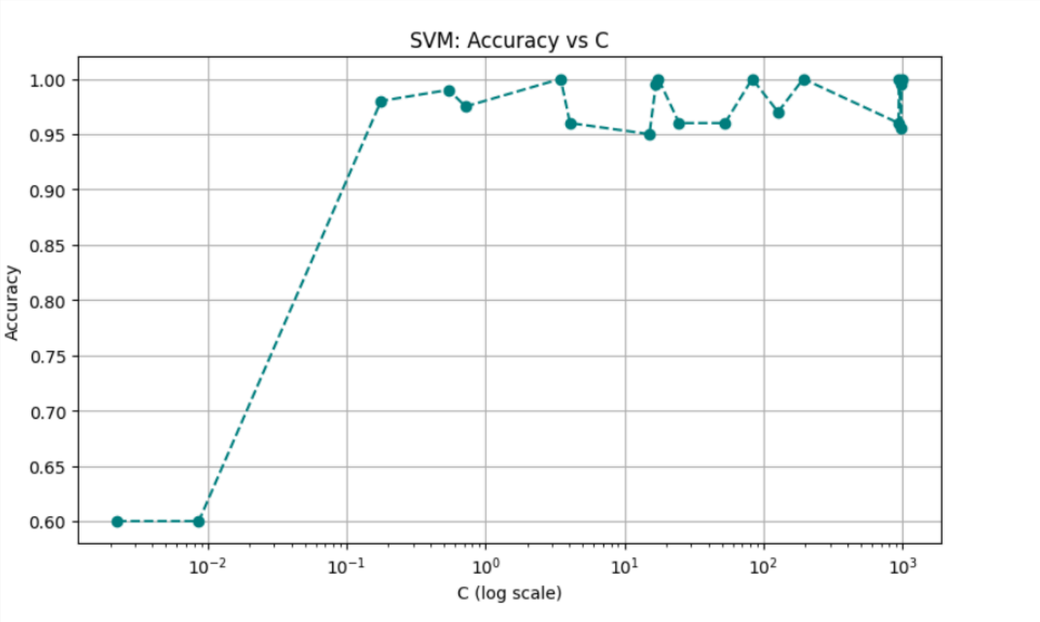
Explanation:

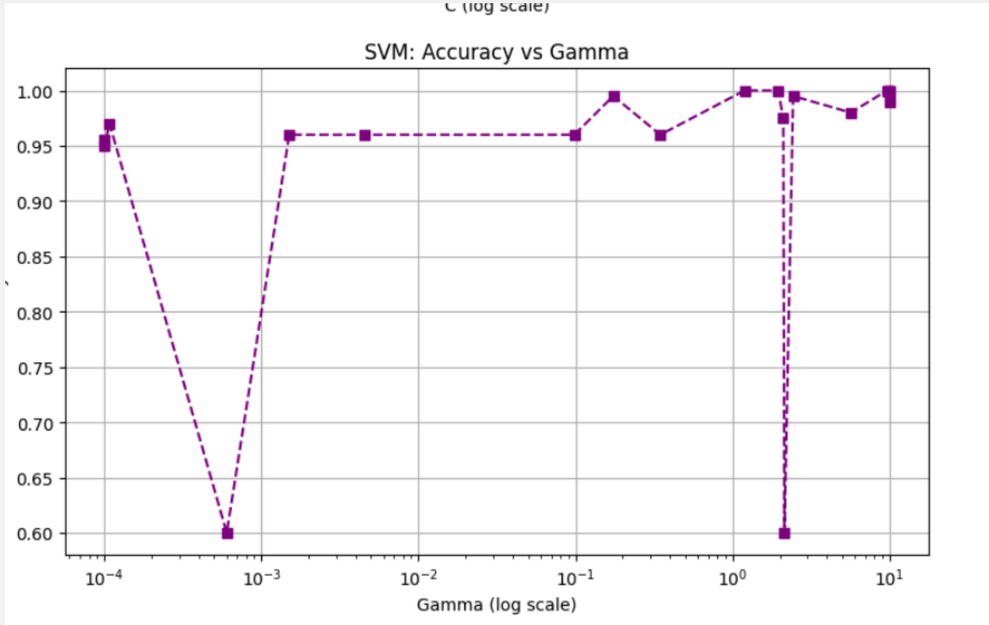
- After the optimization process, the best values for C and gamma are extracted and printed. These are the parameters that give the highest accuracy for the model.

- The final model is trained with the optimal hyperparameters (best_C, best_gamma). Predictions are made on the test set, and the accuracy is evaluated.

C/GAMMA VS ACCURACY

(For Synthetic data)





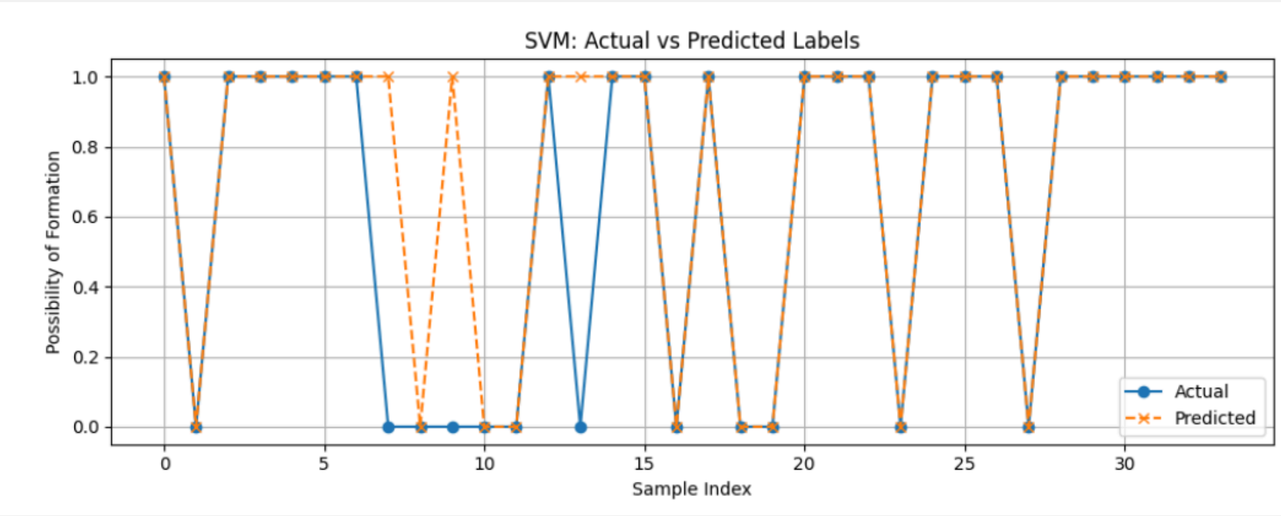
On Synthetic data

Insight:

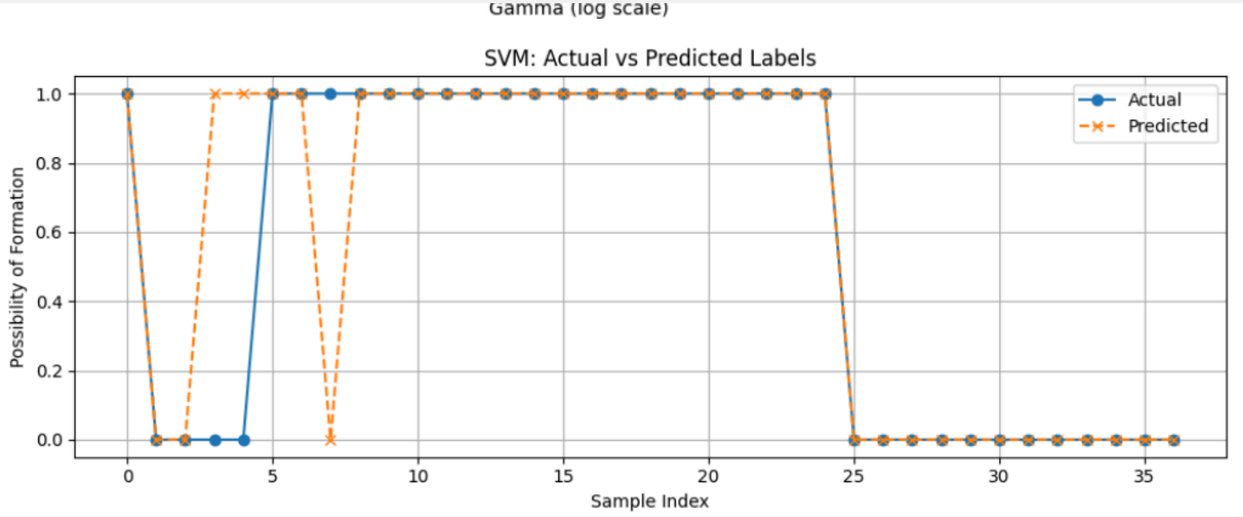
- The best accuracy is achieved when C is around 1000 with an accuracy of approximately 0.96. and stable accuracy is observed for values of C >= 100, maintaining an accuracy of 0.93 or higher.
- The best accuracy is achieved when gamma is around 0.1 with an accuracy of approximately 0.95 and stable accuracy is observed for gamma >= 0.01, maintaining an accuracy of 0.92 or higher.

ACTUAL VS PREDICTED

(For Synthetic data)

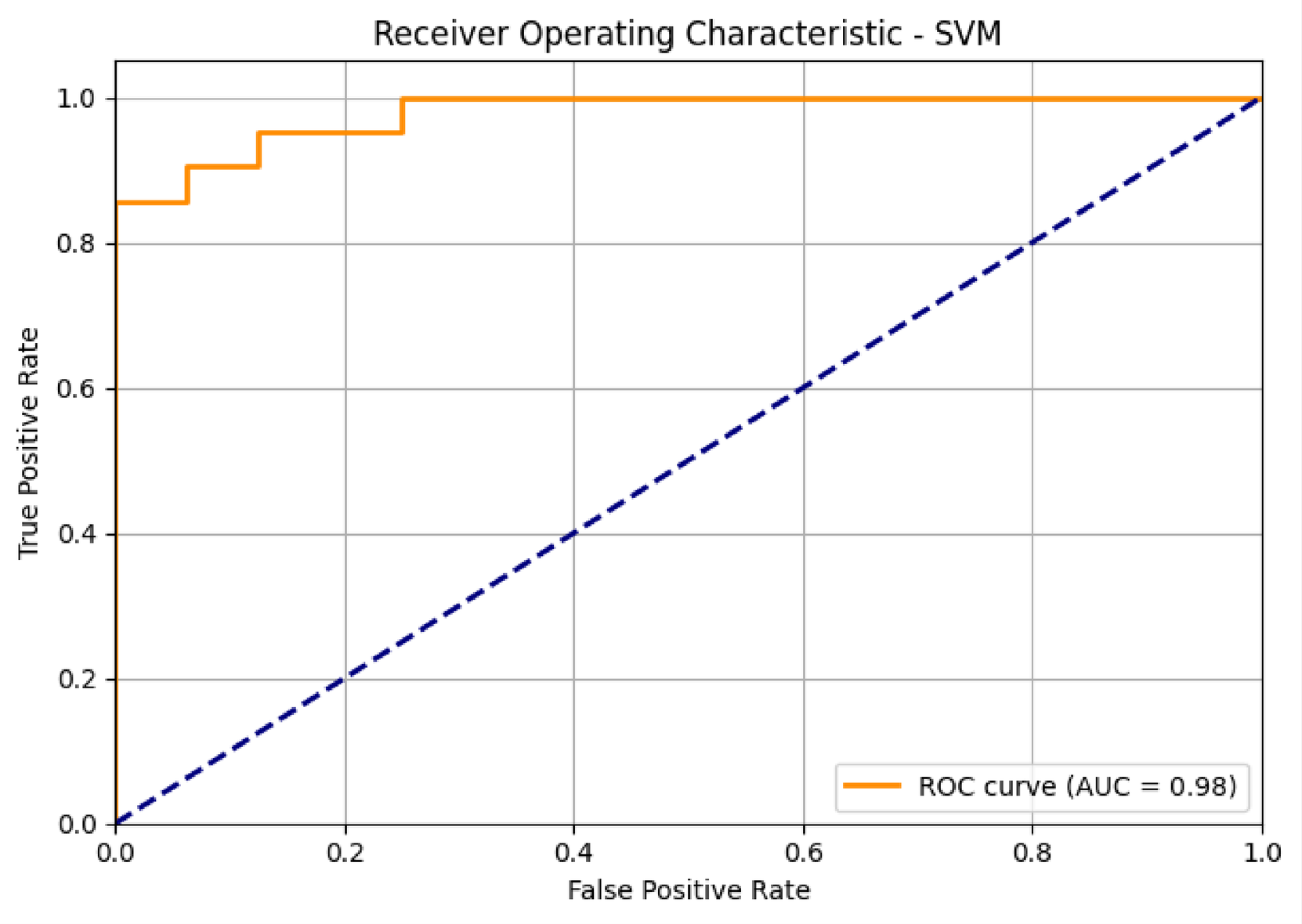


On testing data



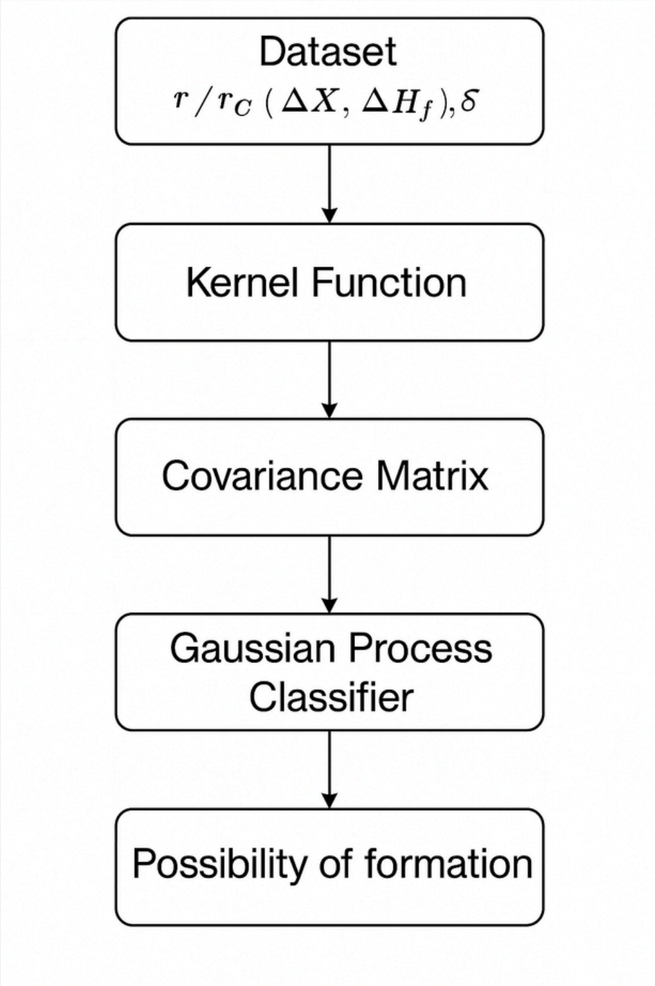
On Suppliment data

SVM ROC Curve for Synthetic Data Model



Gaussian Process Classification ON SYNTHETIC & EXPERIMENTAL DATA

GAUSSIAN PROCESS CLASSIFIER



- In GPC we define a distribution over functions using a Gaussian Process (GP), place a GP prior on a latent function f(x) such that any set of outputs follows a multivariate Gaussian.
- Captures complex, non-linear patterns. Good for small datasets.

DATA STANDARDIZATION AND DEFINING THE GPC MODEL

```
# --- Standardize Features ---
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X)
X_test_scaled = scaler.transform(X_test)
```

```
# Define the GPC with an RBF kernel
kernel = C(1.0) * RBF(length_scale=1.0)
gpc = GaussianProcessClassifier(kernel=kernel, random_state=42)
```

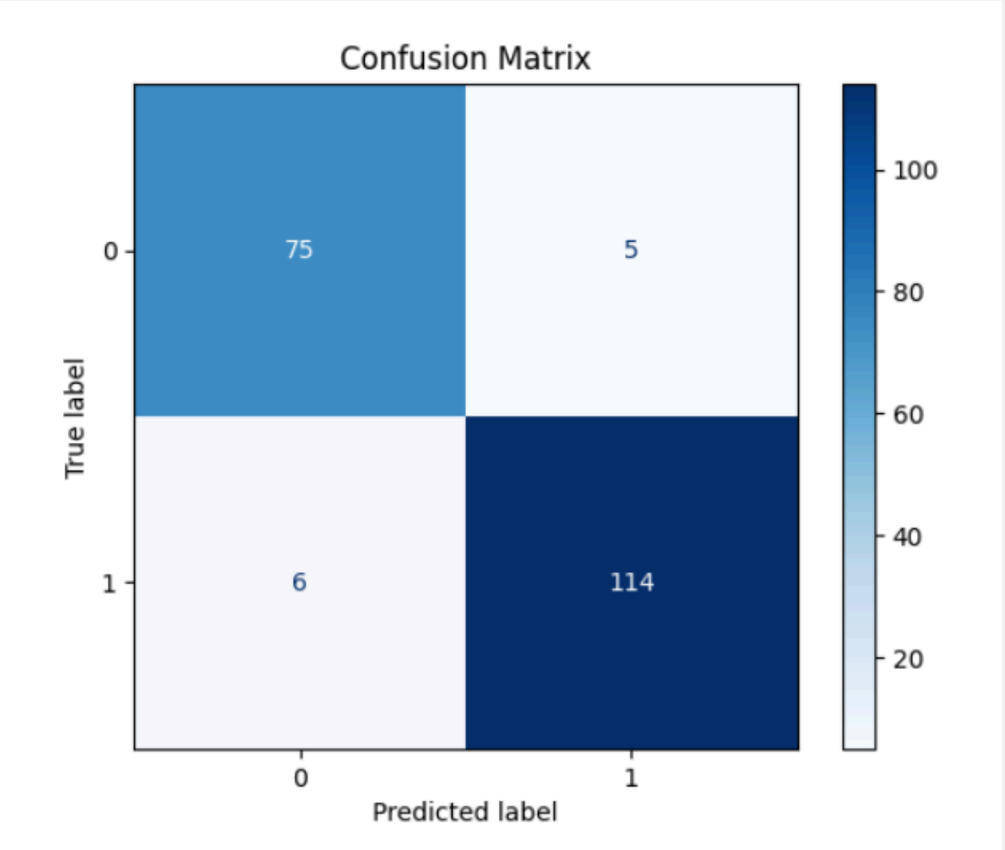
Explanation:

- To ensure all features are on the same scale, the data is standardized using StandardScaler from sklearn. This step improves the performance of many machine learning algorithms, including SVMs.
- We define the Gaussian Process Classifier with a composite kernel: a constant multiplied by the RBF kernel. This allows flexible modeling of non-linear data.

CONFUSION MATRIX FOR GPC MODEL

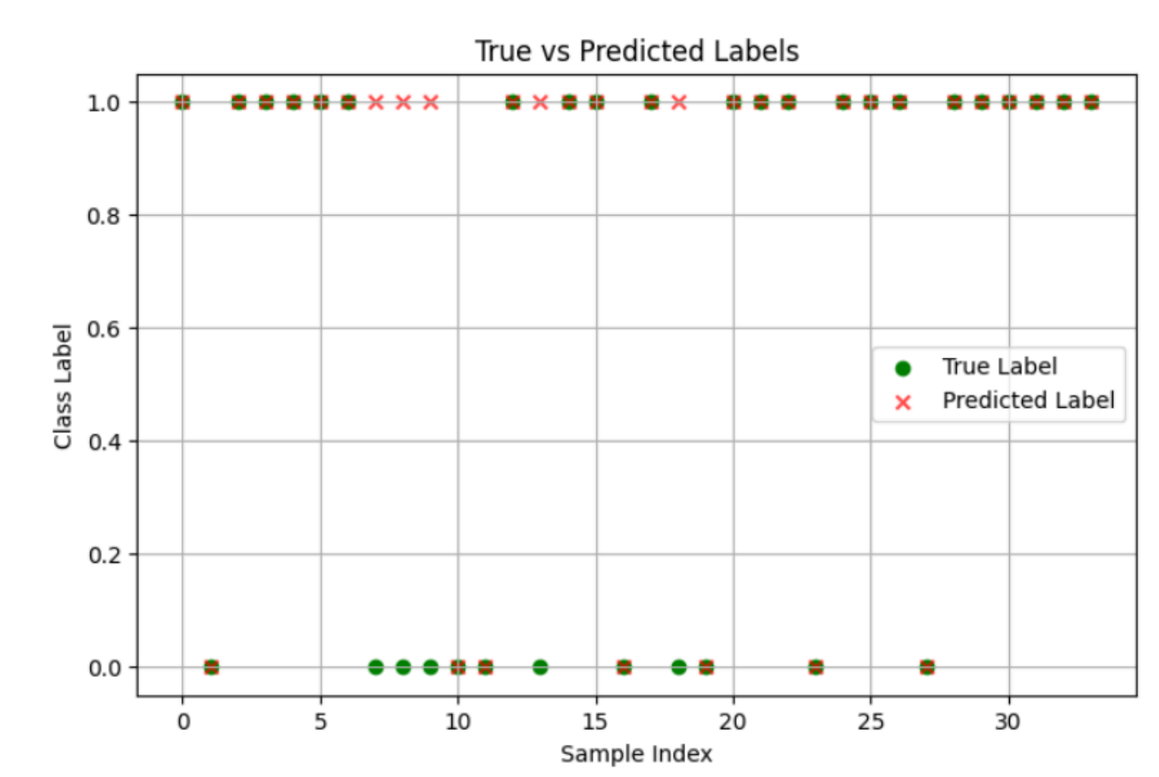
Classification Report:

	precision	recall	f1-score	support
0	0.93	0.94	0.93	80
1	0.96	0.95	0.95	120
accuracy			0.94	200
macro avg	0.94	0.94	0.94	200
weighted avg	0.95	0.94	0.95	200

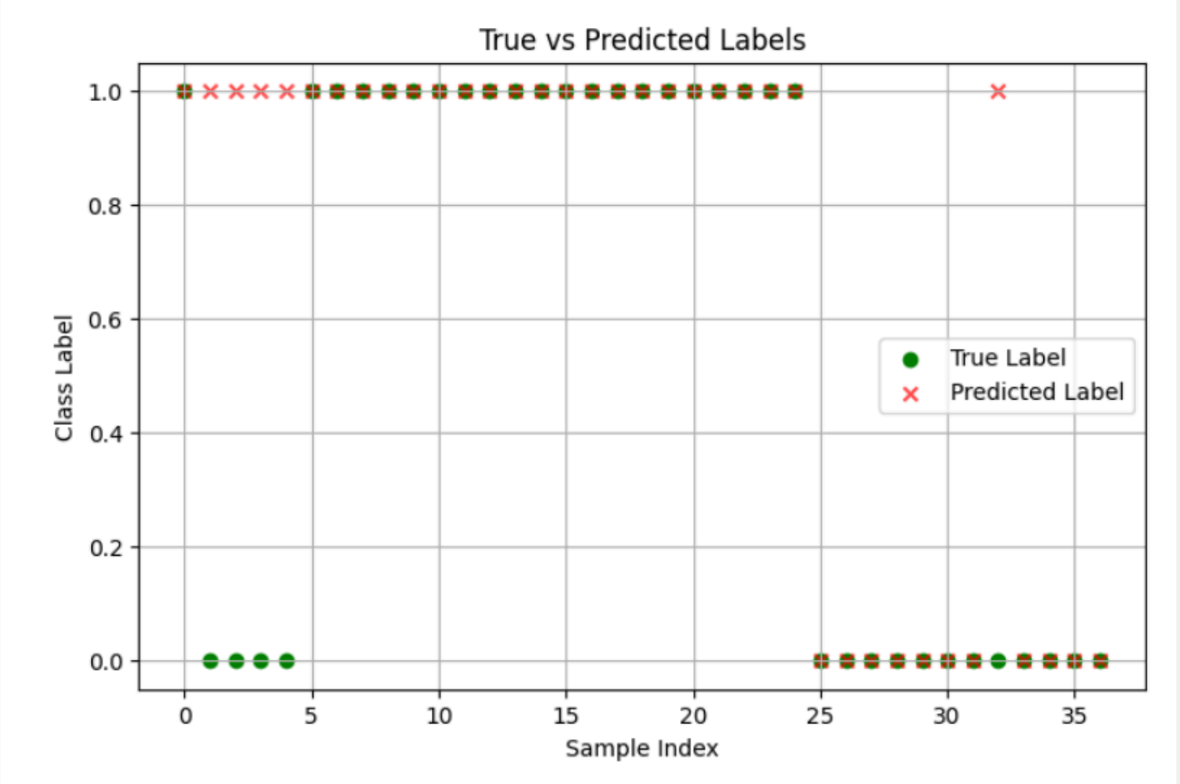


ACTUAL VS PREDICTED

(For Synthetic data)



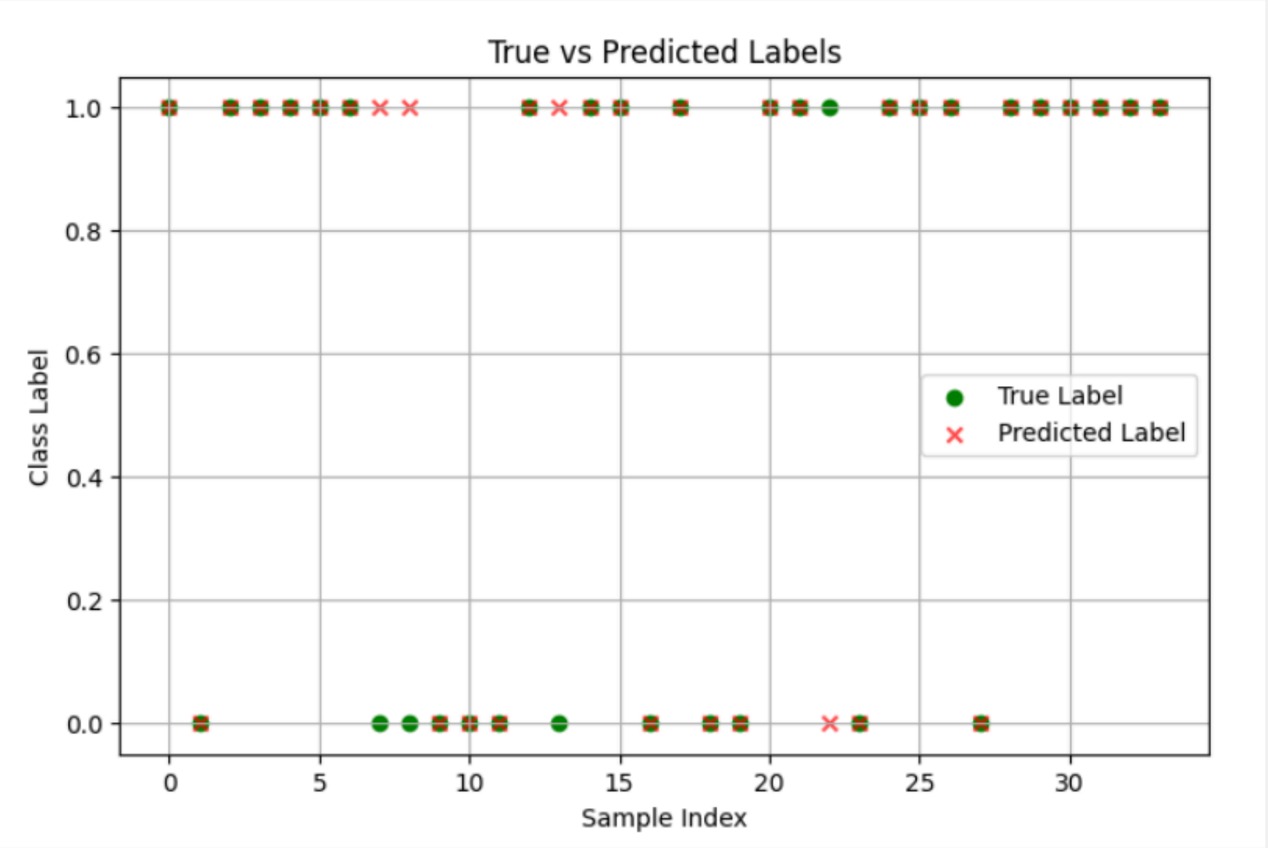
On testing data



On Suppliment data

ACTUAL VS PREDICTED

(For Training data)



On testing data

Final Comparision(Test on Supplement data)

	Original Data		Synthetic Data	
	Accuracy	F1 -Score	Accuracy	F1-Score
Random Forest	0.973	0.98	0.84	0.89
SVM	0.93	0.945	0.92	0.93
KNN	0.94	0.94	0.89	0.93
GPC	0.88	0.91	0.86	0.89

SYNTHETIC DATA PAIRPLOT (SVM Model)

PREDICTED DATA PAIRPLOT (SVM Model)

Conclusion

- Synthetic Data Generation: CTAB-GAN successfully modeled complex distributions of key features like rA/rC, Diff-Xmullikan, ΔHf, ΔSmix, and (Δ) misfit.
- Models trained on CTAB-GAN synthetic data generalize well to real test data.
- SVM Model performed best upon training with synthetic data.
- This approach offers a cost-effective method for accelerating material design and discovery in high entropy borides.
- Overall Project code: https://github.com/arvindd22/UGP

References –

- CTAB GANs Research Paper
- Machine learning based approach for phase prediction in high entropy borides